

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent Application Publication

20250287021

Kind Code

A1

Publication Date

September 11, 2025

Inventor(s)

Zhang; Li et al.

CONDITION DEPENDENT VIDEO BLOCK PARTITION

Abstract

Devices, systems, and methods for video processing are described. In a representative aspect, a method for video processing is disclosed. The method includes determining, for a current video block, whether a first partition mode is applicable to the current video block in responsive to at least one condition. The current video block is split into M sub-blocks in the first partition mode, and $M > 4$. The method also includes performing a conversion for the current video block based on the determination.

Inventors: Zhang; Li (San Diego, CA), Zhang; Kai (San Diego, CA), Liu; Hongbin (Beijing, CN), Xu; Jizheng (San Diego, CA), Wang; Yue (Beijing, CN)

Applicant: Beijing Bytedance Network Technology Co., Ltd. (Beijing, CN); Bytedance Inc. (Los Angeles, CA)

Family ID: 1000008629077

Appl. No.: 19/219597

Filed: May 27, 2025

Foreign Application Priority Data

WO PCT/CN2019/074762

Feb. 11, 2019

WO PCT/CN2019/077161

Mar. 06, 2019

Related U.S. Application Data

parent US continuation 17398800 20210810 PENDING child US 19219597

parent WO continuation PCT/CN2020/074746 20200211 PENDING child US 17398800

Publication Classification

**Int. Cl.: H04N19/176 (20140101); H04N19/119 (20140101); H04N19/157 (20140101);
H04N19/174 (20140101); H04N19/66 (20140101); H04N19/96 (20140101)**

U.S. Cl.:

**CPC H04N19/176 (20141101); H04N19/119 (20141101); H04N19/157 (20141101);
H04N19/174 (20141101); H04N19/66 (20141101); H04N19/96 (20141101);**

Background/Summary

CROSS REFERENCE TO RELATED APPLICATIONS [0001] This application is a continuation of U.S. patent application Ser. No. 17/398,800, filed on Aug. 10, 2021, which is a continuation of International Application No. PCT/CN2020/074746, filed on Feb. 11, 2020, which claims the priority to and benefits of International Patent Applications PCT/CN2019/074762, filed on Feb. 11, 2019 and PCT/CN2019/077161, filed on Mar. 6, 2019. The entire disclosure of the aforementioned applications is incorporated by reference as part of the disclosure of this application.

TECHNICAL FIELD

[0002] The present disclosure relates to video coding techniques, devices and systems.

BACKGROUND

[0003] In spite of the advances in video compression, digital video still accounts for the largest bandwidth use on the internet and other digital communication networks. As the number of connected user devices capable of receiving and displaying video increases, it is expected that the bandwidth demand for digital video usage will continue to grow.

SUMMARY

[0004] Devices, systems and methods related to digital video coding, and specifically, to quinary tree partitioning in video coding are described. The described methods may be applied to both the existing video coding standards (e.g., High Efficiency Video Coding (HEVC)) and future video coding standards (e.g., Versatile Video Coding (VVC)) or codecs.

[0005] In one representative aspect, there is disclosed a method for video processing, comprising: determining, for a current video block, whether a first partition mode is applicable to the current video block in responsive to at least one condition, wherein the current video block is split into M sub-blocks in the first partition mode, and $M > 4$; and performing a conversion for the current video block based on the determination.

[0006] In another representative aspect, there is disclosed a method for video processing, comprising: determining, for a video block, whether and/or how to apply a first partition mode to the video block based on an indication, wherein the video block is split into M portions in the first partition mode, $M > 4$; and performing a conversion of the video block based on the determination.

[0007] In another representative aspect, the above-described method is embodied in the form of processor-executable code and stored in a computer-readable program medium.

[0008] In yet another representative aspect, a device that is configured or operable to perform the above-described method is disclosed. The device may include a processor that is programmed to implement this method.

[0009] In yet another representative aspect, a video decoder apparatus may implement a method as described herein.

[0010] The above and other aspects and features of the disclosed technology are described in greater detail in the drawings, the description and the claims.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

- [0011] FIG. 1 shows examples of macroblock partitions in H.264/Advanced Video Coding (AVC).
- [0012] FIG. 2 shows examples of modes for splitting a coding block into prediction blocks.
- [0013] FIGS. 3A and 3B show examples of partitioning a coding tree block (CTB) and its corresponding quadtree, respectively.
- [0014] FIG. 4 shows an example of a quadtree plus binary tree (QTBT) structure.
- [0015] FIGS. 5A-5F show examples of allowed partitions in VVC.
- [0016] FIGS. 6A-6E show examples of allowed partitions between a parent split (solid) and a current split (dashed), with “X” denoting a disallowed partition.
- [0017] FIGS. 7A and 7B show examples of extended quad-tree (EQT) horizontal and vertical modes, respectively.
- [0018] FIG. 8 shows an example of a signaling structure of QTBT plus EQT partitioning.
- [0019] FIGS. 9A-9H show examples of unsymmetrical quad-tree (UQT) partitioning.
- [0020] FIGS. 10A-10E show examples of quinary tree (QUI-T) partitioning.
- [0021] FIGS. 11A and 11B show examples of senary-partition structures.
- [0022] FIG. 12 is a block diagram illustrating an example of an apparatus that can implement a video encoder and/or decoder, which can be used to implement various portions of the presently disclosed technology.
- [0023] FIG. 13 shows a flowchart of an example method for video processing in accordance with the disclosed technology.
- [0024] FIG. 14 shows a flowchart of another example method for video processing in accordance with the disclosed technology.

DETAILED DESCRIPTION

[0025] Due to the increasing demand of higher resolution video, video coding methods and techniques are ubiquitous in modern technology. Video codecs typically include an electronic circuit or software that compresses or decompresses digital video, and are continually being improved to provide higher coding efficiency. A video codec converts uncompressed video to a compressed format or vice versa. There are complex relationships between the video quality, the amount of data used to represent the video (determined by the bit rate), the complexity of the encoding and decoding algorithms, sensitivity to data losses and errors, ease of editing, random access, and end-to-end delay (latency). The compressed format usually conforms to a standard video compression specification, e.g., the High Efficiency Video Coding (HEVC) standard (also known as H.265 or MPEG-H Part 2), the Versatile Video Coding (VVC) standard to be finalized, or other current and/or future video coding standards.

[0026] Embodiments of the disclosed technology may be applied to existing video coding standards (e.g., HEVC, H.265) and future standards to improve runtime performance. Section headings are used in the present disclosure to improve readability of the description and do not in any way limit the discussion or the embodiments (and/or implementations) to the respective sections only.

1. Overview of Video Coding Standards

[0027] Video coding standards have evolved primarily through the development of the well-known International Telecommunication Union-Telecommunication Standardization Sector (ITU-T) and International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC) standards. The ITU-T produced H.261 and H.263, ISO/IEC produced Moving Picture Experts Group (MPEG)-1 and MPEG-4 Visual, and the two organizations jointly produced the H.262/MPEG-2 Video and H.264/MPEG-4 Advanced Video Coding (AVC) and H.265/HEVC standards. Since H.262, the video coding standards are based on the hybrid video coding structure wherein temporal prediction plus transform coding are utilized. To explore the future video coding technologies beyond HEVC, Joint Video Exploration Team (JVET) was founded by Video Coding

Experts Group (VCEG) and MPEG jointly in 2015. Since then, many new methods have been adopted by JVET and put into the reference software named Joint Exploration Model (JEM). In April 2018, the JVET between VCEG (Q6/16) and ISO/IEC Joint Technical Committee (JTC1) subcommittee (SC) 29/working group (WG) 11 (MPEG) was created to work on the VVC standard targeting at 50% bitrate reduction compared to HEVC.

2. Exemplary Embodiments for Quinary Tree Partitioning

2.1 Partition Tree Structure in H.264/AVC

[0028] The terminology used in H.264/AVC is macroblock (MB) and MB-mode/8×8-mode (partition). Macroblock is the unit wherein each picture/slice is split to and where intra/inter mode decision is applied. And partition defines the level wherein motion information is signaled.

[0029] The core of the coding layer in H.264/AVC was the macroblock, containing a 16×16 block of luma samples and, in the usual case of 4:2:0 color sampling, two corresponding 8×8 blocks of chroma samples.

2.1.1 H.264/AVC Main Profile

[0030] An intra-coded block uses spatial prediction to exploit spatial correlation among pixels. Two partitions are defined: 16×16 and 4×4.

[0031] An inter-coded block uses temporal prediction, instead of spatial prediction, by estimating motion among pictures. Motion can be estimated independently for either 16×16 macroblock or any of its macroblock partitions: 16×8, 8×16, 8×8. A syntax element (MB-mode) is signaled to indicate whether 16×16, 16×8, 8×16 or 8×8 is chosen. If 8×8 is selected, another syntax element (8×8-mode) is further signaled to indicate whether 8×8, 8×4, 4×8, 4×4 (see, e.g., FIG. 1) is used. Only one motion vector (MV) per partition is allowed.

2.1.2 H.264/AVC High Profile

[0032] In the high profile, 8×8 transform and I_8×8 (8×8 intra prediction) is introduced. For intra-coded macroblock, the transform size is fixed, I_16×6 and I_4×4 uses 4×4 transform; I_8×8 uses 8×8 transform.

[0033] For inter-coded macroblocks, either 4×4 or 8×8 transform could be selected. However, the transform size couldn't cross the partition size. For example, if one macroblock chooses 8×8 partition and further selects 8×4 sub-mode, only 4×4 transform may be applied. If one macroblock chooses 16×16, 16×8, 8×16 8×8 partition with 8×8 sub-mode, then either 4×4 or 8×8 transform could be selected.

2.1.3 Summary

[0034] Mode selection is decided in macroblock-level. Transform size shall be no larger than the partition sizes.

2.2 Partition Tree Structure in HEVC

[0035] In HEVC, a coding tree unit (CTU, a.k.a., largest coding unit (LCU)) is split into coding units (CUs) by using a quadtree structure denoted as coding tree to adapt to various local characteristics. The decision whether to code a picture area using inter-picture (temporal) or intra-picture (spatial) prediction is made at the coding unit (CU) level. Each CU can be further split into one, two or four prediction units (PUs) according to the PU splitting type. Inside one PU, the same prediction process is applied and the relevant information is transmitted to the decoder on a PU basis. After obtaining the residual block by applying the prediction process based on the PU splitting type, a CU can be partitioned into transform units (TUs) according to another quadtree structure similar to the coding tree for the CU. One of key feature of the HEVC structure is that it has the multiple partition conceptions including CU, PU, and TU.

[0036] In the following, the various features involved in hybrid video coding using HEVC are highlighted as follows.

[0037] 1) Coding tree units and coding tree block (CTB) structure: The analogous structure in HEVC is the coding tree unit (CTU), which has a size selected by the encoder and can be larger than a traditional macroblock. The CTU consists of a luma CTB and the corresponding chroma

CTBs and syntax elements. The size $L \times L$ of a luma CTB can be chosen as $L=16, 32$, or 64 samples, with the larger sizes typically enabling better compression. HEVC then supports a partitioning of the CTBs into smaller blocks using a tree structure and quadtree-like signaling.

[0038] 2) Coding units (CUs) and coding blocks (CBs): The quadtree syntax of the CTU specifies the size and positions of its luma and chroma CBs. The root of the quadtree is associated with the CTU. Hence, the size of the luma CTB is the largest supported size for a luma CB. The splitting of a CTU into luma and chroma CBs is signaled jointly. One luma CB and ordinarily two chroma CBs, together with associated syntax, form a coding unit (CU). A CTB may contain only one CU or may be split to form multiple CUs, and each CU has an associated partitioning into prediction units (PUs) and a tree of transform units (TUs).

[0039] 3) Prediction units (PUs) and prediction blocks (PBs): The decision whether to code a picture area using inter picture or intra picture prediction is made at the CU level. A PU partitioning structure has its root at the CU level. Depending on the basic prediction-type decision, the luma and chroma CBs can then be further split in size and predicted from luma and chroma prediction blocks (PBs). HEVC supports variable PB sizes from 64×64 down to 4×4 samples. FIG. 2 depicts the allowed PBs.

[0040] 4) Transform units (TUs) and transform blocks: The prediction residual is coded using block transforms. A TU tree structure has its root at the CU level. The luma CB residual may be identical to the luma transform block (TB) or may be further split into smaller luma TBs. The same applies to the chroma TBs. Integer basis functions similar to those of a discrete cosine transform (DCT) are defined for the square TB sizes 4×4 , 8×8 , 16×16 , and 32×32 . For the 4×4 transform of luma intra picture prediction residuals, an integer transform derived from a form of discrete sine transform (DST) is alternatively specified.

2.2.1 Depth of Quadtree

[0041] For a given luma CB of size $M \times M$, a flag signals whether it is split into four blocks of size $M/2 \times M/2$. If further splitting is possible, as signaled by a maximum depth of the residual quadtree indicated in the SPS, each quadrant is assigned a flag that indicates whether it is split into four quadrants. The leaf node blocks resulting from the residual quadtree are the transform blocks that are further processed by transform coding. The encoder indicates the maximum and minimum luma TB sizes that it will use. Splitting is implicit when the CB size is larger than the maximum TB size. Not splitting is implicit when splitting would result in a luma TB size smaller than the indicated minimum. The chroma TB size is half the luma TB size in each dimension, except when the luma TB size is 4×4 , in which case a single 4×4 chroma TB is used for the region covered by four 4×4 luma TBs. In the case of intrapicture-predicted CUs, the decoded samples of the nearest-neighbor TBs (within or outside the CB) are used as reference data for intrapicture prediction.

2.2.2 Summary

[0042] One CTU may be recursively split into multiple CUs based on increased depth of quadtree (e.g., FIG. 3B). Only square CB and TB partitioning is specified, where a block can be recursively split into quadrants, as illustrated in FIG. 3A.

[0043] Mode selection is decided in CU-level. Side information according to a selected mode is signaled in PU-level, such as motion information, intra prediction modes. Residual are signaled in TU-level.

[0044] One PU shall be no larger than CU for inter-coded blocks and one PU shall be equal to CU for intra-coded blocks.

[0045] TU could cross PU for inter-coded blocks, but shall be equal to PU for intra-coded blocks.

2.3 Quadtree Plus Binary Tree Block Structure with Larger CTUs in JEM

[0046] To explore the future video coding technologies beyond HEVC, Joint Video Exploration Team (JVET) was founded by VCEG and MPEG jointly in 2015. Since then, many new methods have been adopted by JVET and put into the reference software named Joint Exploration Model (JEM).

2.3.1 QTBT Block Partitioning Structure

[0047] Different from HEVC, the QTBT structure removes the separation of the CU, PU and TU concepts, and supports more flexibility for CU partition shapes. In the QTBT block structure, a CU can have either a square or rectangular shape. In an example, a coding tree unit (CTU) is first partitioned by a quadtree structure. The quadtree leaf nodes are further partitioned by a binary tree structure. There are two splitting types, symmetric horizontal splitting and symmetric vertical splitting, in the binary tree splitting. The binary tree leaf nodes are called coding units (CUs), and that segmentation is used for prediction and transform processing without any further partitioning. This means that the CU, PU and TU have the same block size in the QTBT coding block structure. In the JEM, a CU sometimes consists of coding blocks (CBs) of different color components, e.g. one CU contains one luma CB and two chroma CBs in the case of P and B slices of the 4:2:0 chroma format and sometimes consists of a CB of a single component, e.g., one CU contains only one luma CB or just two chroma CBs in the case of I slices.

[0048] The following parameters are defined for the QTBT partitioning scheme. [0049] CTU size: the root node size of a quadtree, the same concept as in HEVC [0050] MinQTSIZE: the minimum allowed quadtree leaf node size [0051] MaxBTSIZE: the maximum allowed binary tree root node size [0052] MaxBTDepth: the maximum allowed binary tree depth [0053] MinBTSIZE: the minimum allowed binary tree leaf node size

[0054] In one example of the QTBT partitioning structure, the CTU size is set as 128×128 luma samples with two corresponding 64×64 blocks of chroma samples, the MinQTSIZE is set as 16×16 , the MaxBTSIZE is set as 64×64 , the MinBTSIZE (for both width and height) is set as 4×4 , and the MaxBTDepth is set as 4. The quadtree partitioning is applied to the CTU first to generate quadtree leaf nodes. The quadtree leaf nodes may have a size from 16×16 (i.e., the MinQTSIZE) to 128×128 (i.e., the CTU size). If the leaf quadtree node is 128×128 , it will not be further split by the binary tree since the size exceeds the MaxBTSIZE (i.e., 64×64). Otherwise, the leaf quadtree node could be further partitioned by the binary tree. Therefore, the quadtree leaf node is also the root node for the binary tree and it has the binary tree depth as 0. When the binary tree depth reaches MaxBTDepth (i.e., 4), no further splitting is considered. When the binary tree node has width equal to MinBTSIZE (i.e., 4), no further horizontal splitting is considered. Similarly, when the binary tree node has height equal to MinBTSIZE, no further vertical splitting is considered. The leaf nodes of the binary tree are further processed by prediction and transform processing without any further partitioning. In the JEM, the maximum CTU size is 256×256 luma samples.

[0055] FIG. 4 (left) illustrates an example of block partitioning by using QTBT, and FIG. 4 (right) illustrates the corresponding tree representation. The solid lines indicate quadtree splitting and dotted lines indicate binary tree splitting. In each splitting (i.e., non-leaf) node of the binary tree, one flag is signaled to indicate which splitting type (i.e., horizontal or vertical) is used, where 0 indicates horizontal splitting and 1 indicates vertical splitting. For the quadtree splitting, there is no need to indicate the splitting type since quadtree splitting always splits a block both horizontally and vertically to produce 4 sub-blocks with an equal size.

[0056] In addition, the QTBT scheme supports the ability for the luma and chroma to have a separate QTBT structure. Currently, for P and B slices, the luma and chroma CTBs in one CTU share the same QTBT structure. However, for I slices, the luma CTB is partitioned into CUs by a QTBT structure, and the chroma CTBs are partitioned into chroma CUs by another QTBT structure. This means that a CU in an I slice consists of a coding block of the luma component or coding blocks of two chroma components, and a CU in a P or B slice consists of coding blocks of all three color components.

[0057] In HEVC, inter prediction for small blocks is restricted to reduce the memory access of motion compensation, such that bi-prediction is not supported for 4×8 and 8×4 blocks, and inter prediction is not supported for 4×4 blocks. In the QTBT of the JEM, these restrictions are removed.

2.3.2 Summary of QTBT

[0058] One CTU may be recursively split into multiple CUs based on increased depth of quadtree or binary tree. Square and rectangular CB (with width/height equal to $\frac{1}{2}$ or 2) is specified.

[0059] Mode selection is decided in CU-level. PU and TU are always equal to CU.

2.4 Multiple Type Trees (MTT) for VVC

2.4.1 Proposal in JVET-D0117

[0060] It is proposed that tree types other than quad-tree and binary-tree are supported. In the implementation, two more ternary tree (TT) partitions, i.e., horizontal and vertical center-side triple-trees are introduced, as shown in FIG. 5E and FIG. 5F.

[0061] In some embodiments, the one partition in binary tree (BT)/TT may be further split with BT/TT. Therefore, rectangular blocks are allowed.

[0062] There are two levels of trees, region tree (quad-tree) and prediction tree (binary-tree or triple-tree). A CTU is firstly partitioned by region tree (RT). A RT leaf may be further split with prediction tree (PT). A PT leaf may also be further split with PT until max PT depth is reached. A PT leaf is the basic coding unit. It is still called CU for convenience. A CU cannot be further split. Prediction and transform are both applied on CU in the same way as JEM. The whole partition structure is named 'multiple-type-tree'.

2.4.2 Partition Tree in VVC

[0063] Similarly, it is proposed that three types of partition structures are supported, i.e., quad tree (QT), BT and TT, as shown in the examples in FIGS. 6A-6E. A block split from QT may be further split by QT/BT/TT. a block split from BT or TT may be further split to BT or TT. However, a block split from BT or TT couldn't be further split to QT anymore.

[0064] In VVC, several variables are signaled/derived to control the usage of different partitions. For example:

[0065] maximum multi-type tree depth with offset maxMttDepth for luma and chroma, respectively,

[0066] maximum binary tree size maxBtSize/ternary tree size maxTtSize

[0067] minimum quadtree size MinQtSize/binary tree size MinBtSize/ternary tree size minTtSize

7.3.2.1 Sequence parameter set RBSP syntax

```
TABLE-US-00001 De- scriptor seq_parameter_set_rbsp( ) {    sps_seq_parameter_set_id ue(v) ...
    qtbtt_dual_tree_intra_flag ue(v)    log2_ctu_size_minus2 ue(v)
log2_min_luma_coding_block_size_minus2 ue(v)    partition_constraints_override_enabled_flag
ue(v)    sps_log2_diff_min_qt_min_cb_intra_tile_group_luma ue(v)
sps_log2_diff_min_qt_min_cb_inter_tile_group ue(v)
sps_max_mtt_hierarchy_depth_inter_tile_groups ue(v)
sps_max_mtt_hierarchy_depth_intra_tile_groups_luma ue(v)    if(
sps_max_mtt_hierarchy_depth_intra_tile_groups_luma != 0 ) {
sps_log2_diff_max_bt_min_qt_intra_tile_group_luma ue(v)
sps_log2_diff_max_tt_min_qt_intra_tile_group_luma ue(v)    }    if(
sps_max_mtt_hierarchy_depth_inter_tile_groups != 0 ) {
sps_log2_diff_max_bt_min_qt_inter_tile_group ue(v)
sps_log2_diff_max_tt_min_qt_inter_tile_group ue(v)    }    if( qtbtt_dual_tree_intra_flag ) {
sps_log2_diff_min_qt_min_cb_intra_tile_group_chroma ue(v)
sps_max_mtt_hierarchy_depth_intra_tile_groups_chroma ue(v)    if (
sps_max_mtt_hierarchy_depth_intra_tile_groups_chroma != 0 ) {
sps_log2_diff_max_bt_min_qt_intra_tile_group_chroma ue(v)
sps_log2_diff_max_tt_min_qt_intra_tile_group_chroma ue(v)    }    }    sps_sao_enabled_flag
u(1) ...    rbsp_trailing_bits( ) }
```

Semantics

[0068] sps_max_mtt_hierarchy_depth_inter_tile_groups specifies the default maximum hierarchy depth for coding units resulting from multi-type tree splitting of a quadtree leaf in tile groups with

tile_group_type equal to 0 (B) or 1 (P) referring to the SPS. When partition_constraints_override_flag is equal to 1, the default maximum hierarchy depth can be overridden by tile_group_max_mtt_hierarchy_depth_luma present in the tile group header of the tile groups referring to the SPS. The value of sps_max_mtt_hierarchy_depth_inter_tile_groups shall be in the range of 0 to $\text{CtbLog2SizeY} - \text{MinCbLog2SizeY}$, inclusive.

[0069] sps_max_mtt_hierarchy_depth_intra_tile_groups_luma specifies the default maximum hierarchy depth for coding units resulting from multi-type tree splitting of a quadtree leaf in tile groups with tile_group_type equal to 2 (I) referring to the SPS. When partition_constraints_override_flag is equal to 1, the default maximum hierarchy depth can be overridden by tile_group_max_mtt_hierarchy_depth_luma present in the tile group header of the tile groups referring to the SPS. The value of sps_max_mtt_hierarchy_depth_intra_tile_groups_luma shall be in the range of 0 to $\text{CtbLog2SizeY} - \text{MinCbLog2SizeY}$, inclusive.

[0070] sps_log2_diff_max_bt_min_qt_intra_tile_group_luma specifies the default difference between the base 2 logarithm of the maximum size (width or height) in luma samples of a luma coding block that can be split using a binary split and the minimum size (width or height) in luma samples of a luma leaf block resulting from quadtree splitting of a CTU in tile groups with tile_group_type equal to 2 (I) referring to the SPS. When partition_constraints_override_flag is equal to 1, the default difference can be overridden by tile_group_log2_diff_max_bt_min_qt_luma present in the tile group header of the tile groups referring to the SPS. The value of sps_log2_diff_max_bt_min_qt_intra_tile_group_luma shall be in the range of 0 to $\text{CtbLog2SizeY} - \text{MinQtLog2SizeIntraY}$, inclusive. When sps_log2_diff_max_bt_min_qt_intra_tile_group_luma is not present, the value of sps_log2_diff_max_bt_min_qt_intra_tile_group_luma is inferred to be equal to 0.

[0071] sps_log2_diff_max_tt_min_qt_intra_tile_group_luma specifies the default difference between the base 2 logarithm of the maximum size (width or height) in luma samples of a luma coding block that can be split using a ternary split and the minimum size (width or height) in luma samples of a luma leaf block resulting from quadtree splitting of a CTU in tile groups with tile_group_type equal to 2 (I) referring to the SPS. When partition_constraints_override_flag is equal to 1, the default difference can be overridden by tile_group_log2_diff_max_tt_min_qt_luma present in the tile group header of the tile groups referring to the SPS. The value of sps_log2_diff_max_tt_min_qt_intra_tile_group_luma shall be in the range of 0 to $\text{CtbLog2SizeY} - \text{MinQtLog2SizeIntraY}$, inclusive. When sps_log2_diff_max_tt_min_qt_intra_tile_group_luma is not present, the value of sps_log2_diff_max_tt_min_qt_intra_tile_group_luma is inferred to be equal to 0.

[0072] sps_log2_diff_max_bt_min_qt_inter_tile_group specifies the default difference between the base 2 logarithm of the maximum size (width or height) in luma samples of a luma coding block that can be split using a binary split and the minimum size (width or height) in luma samples of a luma leaf block resulting from quadtree splitting of a CTU in tile groups with tile_group_type equal to 0 (B) or 1 (P) referring to the SPS. When partition_constraints_override_flag is equal to 1, the default difference can be overridden by tile_group_log2_diff_max_bt_min_qt_luma present in the tile group header of the tile groups referring to the SPS. The value of sps_log2_diff_max_bt_min_qt_inter_tile_group shall be in the range of 0 to $\text{CtbLog2SizeY} - \text{MinQtLog2SizeInterY}$, inclusive. When sps_log2_diff_max_bt_min_qt_inter_tile_group is not present, the value of sps_log2_diff_max_bt_min_qt_inter_tile_group is inferred to be equal to 0.

[0073] sps_log2_diff_max_tt_min_qt_inter_tile_group specifies the default difference between the base 2 logarithm of the maximum size (width or height) in luma samples of a luma coding block that can be split using a ternary split and the minimum size (width or height) in luma samples of a luma leaf block resulting from quadtree splitting of a CTU in tile groups with tile_group_type

equal to 0 (B) or 1 (P) referring to the SPS. When `partition_constraints_override_flag` is equal to 1, the default difference can be overridden by `tile_group_log2_diff_max_tt_min_qt_luma` present in the tile group header of the tile groups referring to the SPS. The value of `sps_log2_diff_max_tt_min_qt_inter_tile_group` shall be in the range of 0 to $\text{CtbLog2SizeY} - \text{MinQtLog2SizeInterY}$, inclusive. When `sps_log2_diff_max_tt_min_qt_inter_tile_group` is not present, the value of `sps_log2_diff_max_tt_min_qt_inter_tile_group` is inferred to be equal to 0.

[0074] `sps_log2_diff_min_qt_min_cb_intra_tile_group_chroma` specifies the default difference between the base 2 logarithm of the minimum size in luma samples of a chroma leaf block resulting from quadtree splitting of a chroma CTU with `treeType` equal to `DUAL_TREE_CHROMA` and the base 2 logarithm of the minimum coding block size in luma samples for chroma CUs with `treeType` equal to `DUAL_TREE_CHROMA` in tile groups with `tile_group_type` equal to 2 (I) referring to the SPS. When `partition_constraints_override_flag` is equal to 1, the default difference can be overridden by `tile_group_log2_diff_min_qt_min_cb_chroma` present in the tile group header of the tile groups referring to the SPS. The value of `sps_log2_diff_min_qt_min_cb_intra_tile_group_chroma` shall be in the range of 0 to $\text{CtbLog2SizeY} - \text{MinCbLog2SizeY}$, inclusive. When not present, the value of `sps_log2_diff_min_qt_min_cb_intra_tile_group_chroma` is inferred to be equal to 0. The base 2 logarithm of the minimum size in luma samples of a chroma leaf block resulting from quadtree splitting of a CTU with `treeType` equal to `DUAL_TREE_CHROMA` is derived as follows:

$$\text{MinQtLog2SizeIntraC} = \text{sps_log2_diff_min_qt_min_cb_intra_tile_group_chroma} + \text{MinCbLog2SizeY} \quad (7\ 28)$$

[0075] `sps_max_mtt_hierarchy_depth_intra_tile_groups_chroma` specifies the default maximum hierarchy depth for chroma coding units resulting from multi-type tree splitting of a chroma quadtree leaf with `treeType` equal to `DUAL_TREE_CHROMA` in tile groups with `tile_group_type` equal to 2 (I) referring to the SPS. When `partition_constraints_override_flag` is equal to 1, the default maximum hierarchy depth can be overridden by

`tile_group_max_mtt_hierarchy_depth_chroma` present in the tile group header of the tile groups referring to the SPS. The value of `sps_max_mtt_hierarchy_depth_intra_tile_groups_chroma` shall be in the range of 0 to $\text{CtbLog2SizeY} - \text{MinCbLog2SizeY}$, inclusive. When not present, the value of `sps_max_mtt_hierarchy_depth_intra_tile_groups_chroma` is inferred to be equal to 0.

[0076] `sps_log2_diff_max_bt_min_qt_intra_tile_group_chroma` specifies the default difference between the base 2 logarithm of the maximum size (width or height) in luma samples of a chroma coding block that can be split using a binary split and the minimum size (width or height) in luma samples of a chroma leaf block resulting from quadtree splitting of a chroma CTU with `treeType` equal to `DUAL_TREE_CHROMA` in tile groups with `tile_group_type` equal to 2 (I) referring to the SPS. When `partition_constraints_override_flag` is equal to 1, the default difference can be overridden by `tile_group_log2_diff_max_bt_min_qt_chroma` present in the tile group header of the tile groups referring to the SPS. The value of

`sps_log2_diff_max_bt_min_qt_intra_tile_group_chroma` shall be in the range 0 of to $\text{CtbLog2SizeY} - \text{MinQtLog2SizeIntraC}$, inclusive. When

`sps_log2_diff_max_bt_min_qt_intra_tile_group_chroma` is not the present, value of `sps_log2_diff_max_bt_min_qt_intra_tile_group_chroma` is inferred to be equal to 0.

[0077] `sps_log2_diff_max_tt_min_qt_intra_tile_group_chroma` specifies the default difference between the base 2 logarithm of the maximum size (width or height) in luma samples of a chroma coding block that can be split using a ternary split and the minimum size (width or height) in luma samples of a chroma leaf block resulting from quadtree splitting of a chroma CTU with `treeType` equal to `DUAL_TREE_CHROMA` in tile groups with `tile_group_type` equal to 2 (I) referring to the SPS. When `partition_constraints_override_flag` is equal to 1, the default difference can be

overridden by `tile_group_log2_diff_max_tt_min_qt_chroma` present in the tile group header of the tile groups referring to the SPS. The value of

`sps_log2_diff_max_tt_min_qt_intra_tile_group_chroma` shall be in the range of When 0 to $\text{CtbLog2SizeY} - \text{MinQtLog2SizeIntraC}$, inclusive.

`sps_log2_diff_max_tt_min_qt_intra_tile_group_chroma` is not present, the value of `sps_log2_diff_max_tt_min_qt_intra_tile_group_chroma` is inferred to be equal to 0.

2.4.2.1 Restrictions of Usage of BT and TT

2.4.2.1.1 Variable Definitions

[0078] `tile_group_log2_diff_min_qt_min_cb_luma` specifies the difference between the base 2 logarithm of the minimum size in luma samples of a luma leaf block resulting from quadtree splitting of a CTU and the base 2 logarithm of the minimum coding block size in luma samples for luma CUs in the current tile group. The value of `tile_group_log2_diff_min_qt_min_cb_luma` shall be in the range of 0 to $\text{CtbLog2SizeY} - \text{MinCbLog2SizeY}$, inclusive. When not present, the value of `tile_group_log2_diff_min_qt_min_cb_luma` is inferred as follows: [0079] If `tile_group_type` equal to 2 (I), the value of `tile_group_log2_diff_min_qt_min_cb_luma` is inferred to be equal to `sps_log2_diff_min_qt_min_cb_intra_tile_group_luma` [0080] Otherwise (`tile_group_type` equal to 0 (B) or 1 (P)), the value of `tile_group_log2_diff_min_qt_min_cb_luma` is inferred to be equal to `sps_log2_diff_min_qt_min_cb_inter_tile_group`.

[0081] `tile_group_max_mtt_hierarchy_depth_luma` specifies the maximum hierarchy depth for coding units resulting from multi-type tree splitting of a quadtree leaf in the current tile group. The value of `tile_group_max_mtt_hierarchy_depth_luma` shall be in the range of 0 to $\text{CtbLog2SizeY} - \text{MinCbLog2SizeY}$, inclusive. When not present, the value of `tile_group_max_mtt_hierarchy_depth_luma` is inferred as follows: [0082] If `tile_group_type` equal to 2 (I), the value of `tile_group_max_mtt_hierarchy_depth_luma` is inferred to be equal to `sps_max_mtt_hierarchy_depth_intra_tile_groups_luma` [0083] Otherwise (`tile_group_type` equal to 0 (B) or 1 (P)), the value of `tile_group_max_mtt_hierarchy_depth_luma` is inferred to be equal to `sps_max_mtt_hierarchy_depth_inter_tile_groups`.

[0084] `tile_group_log2_diff_max_bt_min_qt_luma` specifies the difference between the base 2 logarithm of the maximum size (width or height) in luma samples of a luma coding block that can be split using a binary split and the minimum size (width or height) in luma samples of a luma leaf block resulting from quadtree splitting of a CTU in the current tile group. The value of `tile_group_log2_diff_max_bt_min_qt_luma` shall be in the range of 0 to $\text{CtbLog2SizeY} - \text{MinQtLog2SizeY}$, inclusive. When not present, the value of `tile_group_log2_diff_max_bt_min_qt_luma` is inferred as follows: [0085] If `tile_group_type` equal to 2 (I), the value of `tile_group_log2_diff_max_bt_min_qt_luma` is inferred to be equal to `sps_log2_diff_max_bt_min_qt_intra_tile_group_luma` [0086] Otherwise (`tile_group_type` equal to 0 (B) or 1 (P)), the value of `tile_group_log2_diff_max_bt_min_qt_luma` is inferred to be equal to `sps_log2_diff_max_bt_min_qt_inter_tile_group`.

[0087] `tile_group_log2_diff_max_tt_min_qt_luma` specifies the difference between the base 2 logarithm of the maximum size (width or height) in luma samples of a luma coding block that can be split using a ternary split and the minimum size (width or height) in luma samples of a luma leaf block resulting from quadtree splitting of a CTU in in the current tile group. The value of `tile_group_log2_diff_max_tt_min_qt_luma` shall be in the range of 0 to $\text{CtbLog2SizeY} - \text{MinQtLog2SizeY}$, inclusive. When not present, the value of

`tile_group_log2_diff_max_tt_min_qt_luma` is inferred as follows: [0088] If `tile_group_type` equal to 2 (I), the value of `tile_group_log2_diff_max_tt_min_qt_luma` is inferred to be equal to `sps_log2_diff_max_tt_min_qt_intra_tile_group_luma`

[0089] Otherwise (`tile_group_type` equal to 0 (B) or 1 (P)), the value of `tile_group_log2_diff_max_tt_min_qt_luma` is inferred to be equal to `sps_log2_diff_max_tt_min_qt_inter_tile_group`.

[0090] `tile_group_log2_diff_min_qt_min_cb_chroma` specifies the difference between the base 2 logarithm of the minimum size in luma samples of a chroma leaf block resulting from quadtree splitting of a chroma CTU with tree Type equal to DUAL_TREE_CHROMA and the base 2 logarithm of the minimum coding block size in luma samples for chroma CUs with treeType equal to DUAL_TREE_CHROMA in the current tile group. The value of `tile_group_log2_diff_min_qt_min_cb_chroma` shall be in the range of 0 to $\text{CtbLog2SizeY} - \text{MinCbLog2SizeY}$, inclusive. When not present, the value of `tile_group_log2_diff_min_qt_min_cb_chroma` is inferred to be equal to `sps_log2_diff_min_qt_min_cb_intra_tile_group_chroma`.

[0091] `tile_group_max_mtt_hierarchy_depth_chroma` specifies the maximum hierarchy depth for coding units resulting from multi-type tree splitting of a quadtree leaf with treeType equal to DUAL_TREE_CHROMA in the current tile group. The value of `tile_group_max_mtt_hierarchy_depth_chroma` shall be in the range of 0 to $\text{CtbLog2SizeY} - \text{MinCbLog2SizeY}$, inclusive. When not present, the values of `tile_group_max_mtt_hierarchy_depth_chroma` is inferred to be equal to `sps_max_mtt_hierarchy_depth_intra_tile_groups_chroma`.

[0092] `tile_group_log2_diff_max_bt_min_qt_chroma` specifies the difference between the base 2 logarithm of the maximum size (width or height) in luma samples of a chroma coding block that can be split using a binary split and the minimum size (width or height) in luma samples of a chroma leaf block resulting from quadtree splitting of a chroma CTU with treeType equal to DUAL_TREE_CHROMA in the current tile group. The value of `tile_group_log2_diff_max_bt_min_qt_chroma` shall be in the range of 0 to $\text{CtbLog2SizeY} - \text{MinQtLog2SizeC}$, inclusive. When not present, the value of `tile_group_log2_diff_max_bt_min_qt_chroma` is inferred to be equal to `sps_log2_diff_max_bt_min_qt_intra_tile_group_chroma`.

[0093] `tile_group_log2_diff_max_tt_min_qt_chroma` specifies the difference between the base 2 logarithm of the maximum size (width or height) in luma samples of a chroma coding block that can be split using a ternary split and the minimum size (width or height) in luma samples of a chroma leaf block resulting from quadtree splitting of a chroma CTU with treeType equal to DUAL_TREE_CHROMA in the current tile group. The value of `tile_group_log2_diff_max_tt_min_qt_chroma` shall be in the range of 0 to $\text{CtbLog2SizeY} - \text{MinQtLog2SizeC}$, inclusive. When not present, the value of `tile_group_log2_diff_max_tt_min_qt_chroma` is inferred to be equal to `sps_log2_diff_max_tt_min_qt_intra_tile_group_chroma`.

[0094] The variables `MinQtLog2SizeY`, `MinQtLog2SizeC`, `MinQtSizeY`, `MinQtSizeC`, `MaxBtSizeY`, `MaxBtSizeC`, `MinBtSizeY`, `MaxTtSizeY`, `MaxTtSizeC`, `MinTtSizeY`, `MaxMttDepthY` and `MaxMttDepthC` are derived as follows:

$$\text{MinQtLog2SizeY} = \text{MinCbLog2SizeY} + \text{tile_group_log2_diff_min_qt_min_cb_luma} \quad (7-33)$$

$$\text{MinQtLog2SizeC} = \text{MinCbLog2SizeY} + \text{tile_group_log2_diff_min_qt_min_cb_chroma} \quad (7-34)$$

$$\text{MinQtSizeY} = 1 \ll \text{MinQtLog2SizeY} \quad (7-35)$$

$$\text{MinQtSizeC} = 1 \ll \text{MinQtLog2SizeC} \quad (7-36)$$

$$\text{MaxBtSizeY} = 1 \ll (\text{MinQtLog2SizeY} + \text{tile_group_log2_diff_max_bt_min_qt_luma}) \quad (7-37)$$

$$\text{MaxBtSizeC} = 1 \ll (\text{MinQtLog2SizeC} + \text{tile_group_log2_diff_max_bt_min_qt_chroma}) \quad (7-38)$$

$$\text{MinBtSizeY}=1<<\text{MinCbLog2SizeY} \quad (7-39)$$

$$\text{MaxTtSizeY}=1<<(\text{MinQtLog2SizeY}+\text{tile_group_log2_diff_max_tt_min_qt_luma}) \quad (7-40)$$

$$\text{MaxTtSizeC}=1<<(\text{MinQtLog2SizeC}+\text{tile_group_log2_diff_max_tt_min_qt_chroma}) \quad (7-41)$$

$$\text{MinTtSizeY}=1<<\text{MinCbLog2SizeY} \quad (7-42)$$

$$\text{MaxMttDepthY}=\text{tile_group_max_mtt_hierarchy_depth_luma} \quad (7-43)$$

$$\text{MaxMttDepthC}=\text{tile_group_max_mtt_hierarchy_depth_chroma} \quad (7-44)$$

[0095] $\log_2\text{ctu_size_minus2}$, $\log_2\text{min_luma_coding_block_size_minus2}$ are signaled in SPS.

[0096] $\log_2\text{ctu_size_minus2}$ plus 2 specifies the luma coding tree block size of each CTU.

[0097] $\log_2\text{min_luma_coding_block_size_minus2}$ plus 2 specifies the minimum luma coding block size.

[0098] The variables CtbLog2SizeY , CtbSizeY , MinCbLog2SizeY , MinCbSizeY , MinTbLog2SizeY , MaxTbLog2SizeY , MinTbSizeY , MaxTbSizeY , PicWidthInCtbsY , PicHeightInCtbsY , PicSizeInCtbsY , PicWidthInMinCbsY , $\text{PicHeightInMinCbsY}$, PicSizeInMinCbsY , PicSizeInSamplesY , $\text{PicWidthInSamplesC}$ and $\text{PicHeightInSamplesC}$ are derived as follows:

$$\text{CtbLog2SizeY}=\log_2\text{ctu_size_minus2}+2 \quad (7-7)$$

$$\text{CtbSizeY}=1<<\text{CtbLog2SizeY} \quad (7-8)$$

$$\text{MinCbLog2SizeY}=\log_2\text{min_luma_coding_block_size_minus2}+2 \quad (7-9)$$

$$\text{MinCbSizeY}=1<<\text{MinCbLog2SizeY} \quad (7-10)$$

$$\text{MinTbLog2SizeY}=2 \quad (7-11)$$

$$\text{MaxTbLog2SizeY}=6 \quad (7-12)$$

$$\text{MinTbSizeY}=1<<\text{MinTbLog2SizeY} \quad (7-13)$$

$$\text{MaxTbSizeY}=1<<\text{MaxTbLog2SizeY} \quad (7-14)$$

$$\text{PicWidthInCtbsY}=\text{Ceil}(\text{pic_width_in_luma_samples}\div\text{CtbSizeY}) \quad (7-15)$$

$$\text{PicHeightInCtbsY}=\text{Ceil}(\text{pic_height_in_luma_samples}\div\text{CtbSizeY}) \quad (7-16)$$

$$\text{PicSizeInCtbsY}=\text{PicWidthInCtbsY}\times\text{PicHeightInCtbsY} \quad (7-17)$$

$$\text{PicWidthInMinCbsY}=\text{pic_width_in_luma_samples}/\text{MinCbSizeY} \quad (7-18)$$

$$\text{PicHeightInMinCbsY}=\text{pic_height_in_luma_samples}/\text{MinCbSizeY} \quad (7-19)$$

$$\text{PicSizeInMinCbsY}=\text{PicWidthInMinCbsY}\times\text{PicHeightInMinCbsY} \quad (7-20)$$

$$\text{PicSizeInSamplesY}=\text{pic_width_in_luma_samples}\times\text{pic_height_in_luma_samples} \quad (7-21)$$

PicWidthInSamplesC=pic_width_in_luma_samples/SubWidthC (7-22)

PicHeightInSamplesC=pic_height_in_luma_samples/SubHeightC (7-23)

[0099] [Ed. (BB): Currently the maximum transform size (64×64 luma samples and corresponding chroma sample size) and the minimum transform size (4×4 luma samples and corresponding chroma samples) is fixed, pending further specification development.]

2.4.2.1.2 Allowed Binary Split Process

[0100] Inputs to this process are: [0101] a binary split mode btSplit, [0102] a coding block width cbWidth, [0103] a coding block height cbHeight, [0104] a location (x0, y0) of the top-left luma sample of the considered coding block relative to the top-left luma sample of the picture, [0105] a multi-type tree depth mttDepth, [0106] a maximum multi-type tree depth with offset maxMttDepth, [0107] a maximum binary tree size maxBtSize, [0108] a partition index partIdx.

[0109] Output of this process is the variable allowBtSplit.

TABLE-US-00002 TABLE 1 Specification of parallelTtSplit and cbSize based on btSplit
btSplit == SPLIT_BT_VER SPLIT_BT_HOR parallelTtSplit SPLIT_TT_VER
SPLIT_TT_HOR cbSize cbWidth cbHeight

[0110] The variables parallelTtSplit and cbSize are derived as specified in Table 1.

[0111] The variable allowBtSplit is derived as follows: [0112] If one or more of the following conditions are true, allowBtSplit is set equal to FALSE:

[0113] //according to block size and maximum allowed MTT depth [0114] cbSize is less than or equal to MinBtSizeY [0115] cbWidth is greater than maxBtSize [0116] cbHeight is greater than maxBtSize [0117] mttDepth is greater than or equal to maxMttDepth [0118] Otherwise, if all of the following conditions are true, allowBtSplit is set equal to FALSE

[0119] //according to picture boundary (no vertical BT for bottom picture boundary and bottom-right picture boundary) [0120] btSplit is equal to SPLIT_BT_VER [0121] y0+cbHeight is greater than pic_height_in_luma_samples [0122] Otherwise, if all of the following conditions are true, allowBtSplit is set equal to FALSE

[0123] //according to picture boundary (no horizontal BT for right picture boundary) [0124] btSplit is equal to SPLIT_BT_HOR [0125] x0+cbWidth is greater than pic_width_in_luma_samples [0126] y0+cbHeight is less than or equal to pic_height_in_luma_samples [0127] Otherwise, if all of the following conditions are true, allowBtSplit is set equal to FALSE:

[0128] //according to TT partition in above level (mttDepth-1) [0129] mttDepth is greater than 0 [0130] partIdx is equal to 1 [0131] MttSplitMode[x0][y0][mttDepth-1] is equal to parallelTtSplit

[0132] //according to transform sizes (e.g., when MaxTbSizeY is equal to 64, for 64×128, no vertical BT; for 128×64, no horizontal BT) [0133] Otherwise if all of the following conditions are true, allowBtSplit is set equal to FALSE [0134] btSplit is equal to SPLIT_BT_VER [0135]

cbWidth is less than or equal to MaxTbSizeY [0136] cbHeight is greater than MaxTbSizeY [0137] Otherwise if all of the following conditions are true, allowBtSplit is set equal to FALSE [0138] btSplit is equal to SPLIT_BT_HOR [0139] cbWidth is greater than MaxTbSizeY [0140] cbHeight is less than or equal to MaxTbSizeY [0141] Otherwise, allowBtSplit is set equal to TRUE.

2.4.2.1.3 Allowed Ternary Split Process

[0142] Inputs to this process are: [0143] a ternary split mode ttSplit, [0144] a coding block width cbWidth, [0145] a coding block height cbHeight, [0146] a location (x0, y0) of the top-left luma sample of the considered coding block relative to the top-left luma sample of the picture, [0147] a multi-type tree depth mttDepth [0148] a maximum multi-type tree depth with offset maxMttDepth, [0149] a maximum binary tree size maxTtSize.

[0150] Output of this process is the variable allowTtSplit.

TABLE-US-00003 TABLE 2 Specification of cbSize based on ttSplit
ttSplit == SPLIT_TT_VER
ttSplit == SPLIT_TT_HOR cbSize cbWidth cbHeight

[0151] The variable `cbSize` is derived as specified in Table 2.

[0152] The variable `allowTtSplit` is derived as follows: [0153] If one or more of the following conditions are true, `allowTtSplit` is set equal to FALSE:

[0154] //according to block size [0155] `cbSize` is less than or equal to $2 \times \text{MinTtSizeY}$ [0156] `cbWidth` is greater than $\text{Min}(\text{MaxTbSizeY}, \text{maxTtSize})$ [0157] `cbHeight` is greater than $\text{Min}(\text{MaxTbSizeY}, \text{maxTtSize})$

[0158] //according to maximum allowed MTT depth [0159] `mttDepth` is greater than or equal to `maxMttDepth`

[0160] //according to whether it is located at picture boundary [0161] $x_0 + \text{cbWidth}$ is greater than `pic_width_in_luma_samples` [0162] $y_0 + \text{cbHeight}$ is greater than `pic_height_in_luma_samples` [0163] Otherwise, `allowTtSplit` is set equal to TRUE.

2.5 Partition Tree Structure in A VS3

[0164] In AVS3, Extended Quad-tree (EQT) partitioning is adopted, which further extends the QTBT scheme and increases the partitioning flexibility. More specially, EQT splits a parent CU into four sub-CUs of different sizes, which can adequately model the local image content that cannot be elaborately characterized with QTBT. Meanwhile, EQT partitioning allows the interleaving with BT partitioning for enhanced adaptability.

[0165] With the EQT partitioning, a parent CU is split into four sub-CUs with different sizes. As shown in FIG. 7, EQT divides a $M \times N$ parent CU into two $M \times N/4$ CUs and two $M/2 \times N/2$ CUs in the horizontal direction. Analogously, EQT vertical partitioning generates two $N \times M/4$ CUs and two $M/2 \times N/2$ CUs. In particular, EQT sub-blocks size is always the power of 2, such that additional transformations are not necessarily involved.

[0166] In the structure of QTBT, a QT splitting flag is first signaled to indicate whether the current CU is split by QT. As such, when this `ag` is false, the second signal will be encoded to denote whether the current CU splitting mode is non-splitting or BT splitting. For a BT splitting CU, the third bin (DIR) is signaled to discriminate horizontal BT or vertical BT splitting. When EQT partitioning is introduced, one additional bin termed as `isEQT` is signaled to indicate whether it is an EQT-split, in case that BT and EQT are both available, as shown in FIG. 8.

2.6 UQT

[0167] Unsymmetrical Quad-Tree (UQT) partitioning is proposed in our P1809119401H. With UQT, a block with dimensions $W \times H$ is split into four partitions with dimensions $W_1 \times H_1$, $W_2 \times H_2$, $W_3 \times H_3$ and $W_4 \times H_4$, where W_1 , W_2 , W_3 , W_4 , H_1 , H_2 , H_3 , H_4 are all integers. All the parameters are in the form of power of 2. For example, $W_1 = 2N_1$, $W_2 = 2N_2$, $W_3 = 2N_3$, $W_4 = 2N_4$, $H_1 = 2M_1$, $H_2 = 2M_2$, $H_3 = 2M_3$, $H_4 = 2M_4$. Some examples are shown in FIGS. 9A-9F.

3. Drawbacks and Problems in Existing Systems

[0168] Although the QT/BT/TT coding tree structure in VVC is quite flexible, there is still some partitioning patterns that cannot be attained by QT/BT/TT/EQT/UQT.

4. Exemplary Methods for Quinary Tree Partitioning

[0169] To address the problem, several methods are proposed to introduce other kinds of partition structures that may split one block to more than 4 partitions.

[0170] The detailed inventions below should be considered as examples to explain general concepts. These embodiments should not be interpreted in a narrow way. Furthermore, these embodiments can be combined in any manner.

[0171] In the following discussion, partition trees may indicate QT, BT, TT or Unsymmetrical Quad-Tree (UQT), EQT or others. While partition/splitting directions may indicate the horizontal splitting or vertical splitting or diagonal splitting or others. One partition is denoted by its partition tree type and partition direction.

[0172] QT, BT, TT, UQT, or EQT may refer to “QT split”, “BT split”, “TT split”, “UQT split”, “EQT split”, respectively.

[0173] In the following discussion, “split” and “partitioning” have the same meaning. The

proposed methods may be also applicable to existing partition trees.

Definitions of Proposed Partition Types

[0174] 1. Quinary-Tree (QUI-T) partitioning is proposed. With QUI-T, a block with dimensions $W \times H$ is split into five smaller blocks. When the indication of using such partitioning is true, such a block is directly split into five smaller ones (a.k.a. split child blocks). The smaller one may be treated as a coding unit/a prediction unit/a transform unit. Each dimension of the smaller block may be denoted by $W_{\text{sub}.i} \times H_{\text{sub}.i}$ (i being $0 \dots 4$, indicating the partition index) and W_i, H_i are all integers. [0175] a. In one example, each smaller block may be further split into even smaller blocks, such as in a recursive way. [0176] b. In one example, all the dimensions are in the form of power of 2. [0177] i. For example, $W_0=2N_0, W_1=2N_1, W_2=2N_2, W_3=2N_3, W_4=2N_4, H_0=2M_0, H_1=2M_1, H_2=2M_2, H_3=2M_3, H_4=2M_4$. [0178] c. In one example, QUI-T split one block in both horizontal and vertical directions. Such case is named as mixed direction. FIGS. **10A, 10B** and **10D** give some examples. [0179] i. For example, at least one of W_i is unequal to W . [0180] ii. For example, at least one of H_i is unequal to H . [0181] d. In one example, QUI-T only splits one block in vertical direction. [0182] i. For example, $H_0=H_1=H_2=H_3=H_4=H$. FIG. **10C** gives an example. [0183] e. In one example, QUI-T only splits one block in horizontal direction. [0184] i. For example, $W_0=W_1=W_2=W_3=W_4=W$. FIG. **10E** gives an example. [0185] f. In one example, one of the partitions (with size equal $W_{\text{sub}.x} \times H_{\text{sub}.x}$) to has different block sizes compared to others. [0186] i. In one example, the other four partitions are with same sizes. [0187] ii. In one example, $W_{\text{sub}.x}=1/2W$ and $W_{\text{sub}.y}=1/8W$ ($y \neq x$) with x being a value within the range $[0, 4]$. Alternatively, furthermore, x is equal to 1 or 3. [0188] 1. FIG. **10C** gives an example wherein $W_{\text{sub}.3}=1/2W$ and $W_{\text{sub}.y}=1/8W$ ($y \neq 3$). [0189] iii. In one example, $H_{\text{sub}.x}=1/2H$ and $H_{\text{sub}.y}=1/8H$ ($y \neq x$). Alternatively, furthermore, x is equal to 1 or 3. [0190] iv. In one example, $W_{\text{sub}.x}=W - ((W/5) < 2)$ and $W_{\text{sub}.y}=W/5$ ($y \neq x$) with x being a value within the range $[0, 4]$. [0191] v. In one example, $W_{\text{sub}.x}=W - ((W/M) < 2)$ and $W_{\text{sub}.y}=W/M$ ($y \neq x$) wherein M is an integer value, such as 8, 16, 32, 64. [0192] g. In one example, two of the partitions may have equal size (with size equal $W_{\text{sub}.x} \times H_{\text{sub}.x}$), the others are with equal sizes but different from these two. [0193] i. In one example, $W_{\text{sub}.x}=1/8W$ and $W_{\text{sub}.y}=1/4W$ (for all y that $y \neq x$). Alternatively, furthermore, x being equal to 0 and 3. Alternatively, furthermore, x being equal to 0 and 4. Alternatively, furthermore, x being equal to 1 and 3. Alternatively, furthermore, x being equal to 1 and 4. [0194] 1. FIG. **10C** gives an example wherein $W_{\text{sub}.1}=W_{\text{sub}.3}=1/8W$ and $W_{\text{sub}.y}=1/4W$ ($y \neq 3$ & $y \neq 1$). [0195] ii. In one example, $H_{\text{sub}.x}=1/8H$ and $H_{\text{sub}.y}=1/4H$ (for all y that $y \neq x$). Alternatively, furthermore, x being equal to 0 and 3. Alternatively, furthermore, x being equal to 0 and 4. Alternatively, furthermore, x being equal to 1 and 3. Alternatively, furthermore, x being equal to 1 and 4. [0196] h. In one example, two of the partitions may have equal size (with size equal $W_{\text{sub}.x} \times H_{\text{sub}.x}$), the other three may have different sizes. [0197] i. In one example, $W_{\text{sub}.x}=1/16W$ for the two with equal sizes; and $W_{\text{sub}.i}=2/16W$; $W_{\text{sub}.j}=4/16W$ and $W_{\text{sub}.k}=8/16W$, wherein i, j, k are unequal to x . [0198] 1. FIG. **10C** gives an example wherein $W_{\text{sub}.0}=W_{\text{sub}.4}=1/16W$; $W_{\text{sub}.1}=2/16W$; $W_{\text{sub}.2}=4/16W$ and $W_{\text{sub}.3}=8/16W$. [0199] ii. In one example, $H_{\text{sub}.x}=1/16H$ for the two with equal sizes; and $H_{\text{sub}.i}=2/16W$; $H_{\text{sub}.j}=4/16W$ and $H_{\text{sub}.k}=8/16W$, wherein i, j, k are unequal to x . [0200] iii. Alternatively, two of the five partitions may have equal size (with size equal $W_{\text{sub}.x} \times H_{\text{sub}.x}$), two of the remaining three are with the same size. [0201] i. In the above and below description, $1/16W$ or $1/8W$ mean $1/16 \times W$ or $1/8 \times W$, or noted as $W/16$ or $W/8$. [0202] j. In one example, $W_{\text{sub}.0}=W_{\text{sub}.4}=W/8$ and $W_{\text{sub}.1}=W_{\text{sub}.2}=W_{\text{sub}.3}=W/4$, $H_{\text{sub}.0}=H_{\text{sub}.1}=H_{\text{sub}.2}=H_{\text{sub}.3}=H_{\text{sub}.4}=H$. [0203] k. In one example, $H_{\text{sub}.0}=H_{\text{sub}.4}=H/8$ and $H_{\text{sub}.1}=H_{\text{sub}.2}=H_{\text{sub}.3}=H/4$, $W_{\text{sub}.0}=W_{\text{sub}.1}=W_{\text{sub}.2}=W_{\text{sub}.3}=W_{\text{sub}.4}=W$. [0204] l. In one example, one or multiple splits are not allowed for one or more child blocks when the current block is split by quinary-split. [0205] i. For example, if the current block is split by quinary-split into five child-block: $B_0: W/8 \times H$, $B_1: W/4 \times H$, $B_2: W/4 \times H$, $B_3: W/4 \times H$, $B_4: W/8 \times H$, then: [0206] 1. In one example, B_x is not allowed

to be split by vertical BT, where x may be one or some of 0, 1, 2, 3, 4; [0207] 2. In one example, Bx is not allowed to be split by vertical TT, where x may be one or some of 0, 1, 2, 3, 4; [0208] 3. In one example, Bx is not allowed to be split by horizontal BT, where x may be one or some of 0, 1, 2, 3, 4; [0209] 4. In one example, Bx is not allowed to be split by horizontal TT, where x may be one or some of 0, 1, 2, 3, 4; [0210] 5. In one example, Bx is not allowed to be split by QT, where x may be one or some of 0, 1, 2, 3, 4; [0211] ii. For example, if the current block is split by quinary-split into five child-block: B0: $W \times H/8$, B1: $W \times H/4$, B2: $W \times H/4$, B3: $W \times H/4$, B4: $W \times H/8$, then: [0212] 1. In one example, Bx is not allowed to be split by horizontal BT, where x may be one or some of 0, 1, 2, 3, 4; [0213] 2. In one example, Bx is not allowed to be split by horizontal TT, where x may be one or some of 0, 1, 2, 3, 4; [0214] 3. In one example, Bx is not allowed to be split by vertical BT, where x may be one or some of 0, 1, 2, 3, 4; [0215] 4. In one example, Bx is not allowed to be split by vertical TT, where x may be one or some of 0, 1, 2, 3, 4; [0216] 5. In one example, Bx is not allowed to be split by QT, where x may be one or some of 0, 1, 2, 3, 4; [0217] m. The above methods may be extended to other Senary-, Septenary-, Octonary-Tree partitions (SnT, StT, OctT) wherein one block may be split to 6, 7, or 8 smaller blocks. Examples of senary-tree partitions are shown in FIGS. 11A and 11B. [0218] n. Some exemplary partitions are depicted in FIGS. 10A-10E. [0219] o. The coding order (denoted by PIdx 0 . . . 4) may be different from that defined in FIGS. 10A-10E. [0220] i. The coding order for one QUI-T pattern may be pre-defined. [0221] ii. Alternatively, multiple coding orders may be pre-defined for one QUI-T pattern, and one block may choose one from them, such as via signaling the indication of selected coding order or derivation at the decoder side.

Interaction with Other Partition Types [0222] 2. A block which is split into child blocks by QUI-T, may be split from a parent block by one or some specific kinds of split methods. [0223] a. A block which may allow QUI-T partitions, may be a block generated by QT or BT or TT or QUI-T partitions. [0224] b. For example, a block which is split into child blocks by QUI-T, can only be split from a parent block by QT. [0225] c. A block which may allow QUI-T partitions, may be a root block. [0226] 3. A block which is split from a parent block by QUI-T, may be further split into child blocks by one or multiple other partition types (such as QT, BT, TT, QUI-T, UQT). [0227] a. For example, a block which is split from a parent block by QUI-T, may be further split into child blocks by BT and/or TT. [0228] b. For example, a block which is split from a parent block by QUI-T, may be further split into child blocks by BT and/or TT, and/or QUI-T, but not QT. [0229] c. For example, a block which is split from a parent block by QUI-T, may be further split into child blocks by QUI-T and/or QT, but not BT/TT. [0230] d. For example, a block which is split from a parent block by QUI-T, cannot be further split into child blocks by QT. [0231] e. Alternatively, QUI-T split blocks may be not further split into child blocks. [0232] 4. When a parent block is split into child blocks by QUI-T, the split depth of the child block may be derived from the split depth of the parent block. [0233] a. In one example, the splitting due to QUI-T may be used to update the QT/BT/TT/QUI-T/MTT depth. [0234] i. In one example, the QT depth of one or all of the child blocks is equal to the QT depth of the parent block added by 1. [0235] ii. In one example, the BT depth of one or all of the child blocks is equal to the BT depth of the parent block added by 1. [0236] iii. In one example, the TT depth of one or all of the child blocks is equal to the TT depth of the parent block added by 1. [0237] iv. In one example, the QUI-T depth of one or all of the child blocks is equal to the QUI-T depth of the parent block added by 1. [0238] v. In one example, the MTT depth of one or all of the child block is equal to the MTT depth of the parent block added by 1. [0239] 1. For example, the MTT depth of the child block is equal to the MTT depth of the parent block added by 1 if the parent block is split into child blocks by BT. [0240] 2. For example, the MTT depth of the child block is equal to the MTT depth of the parent block added by 1 if the parent block is split into child blocks by TT. [0241] b. In one example, the QUI-T/BT/TT/QT/MTT depth increasement for different child block may be different. [0242] i. The depth increasement is dependent on the ratio of a child block compared to the parent block.

Restrictions of Usage of QUI-T

[0243] 5. In one example, the maximum/minimum block size that could allow QUI-T partitions and/or the maximum bit depth and/or maximum depth that could allow QUI-T partitions may be signaled in sequence parameter set (SPS)/picture parameter set (PPS)/video parameter set (VPS)/adaptation parameter set (APS)/sequence header/picture header/slice header/tile group header/CTU row/regions, etc. [0244] a. The maximum/minimum block size that could allow QUI-T partitions and/or the maximum depth that could allow QUI-T partitions may be derived from other values, such as depth for MTT or depth of QT. [0245] b. The maximum block that allows QUI-T partitions, may be the largest coding block (coding tree block or coding tree unit). [0246] c. For example, the maximum block that allows QUI-T partitions, may be the virtual pipeline data unit (VPDU). [0247] d. In one example, the maximum/minimum block size that could allow QUI-T partitions and/or the maximum depth that could allow QUI-T partitions may be dependent on profile/level/tier of a standard. [0248] e. In one example, the maximum/minimum block size that could allow QUI-T partitions and/or the maximum depth that could allow QUI-T partitions may be derived, such as to be the same as that for QT partitions. [0249] f. In one example, the maximum/minimum block size that could allow QUI-T partitions and/or the maximum depth that could allow QUI-T partitions may be dependent on tile group tile/slice type/color component/dual tree is enabled or not. [0250] g. In one example, the maximum/minimum block size that could allow QUI-T partitions and/or the maximum depth that could allow QUI-T partitions may be different for different QUI-T patterns. [0251] h. When one block is split according to QUI-T, the corresponding depth of QUI-T of one smaller block may be adjusted (e.g., increased by 1) accordingly. [0252] i. Alternatively, the corresponding depth of a certain partition (e.g., QT) of one smaller block may be adjusted (e.g., increased by 1) accordingly. [0253] ii. Alternatively, the corresponding depth of MTT of one smaller block may be adjusted (e.g., increased by 1) accordingly. [0254] iii. The adjustment of corresponding depth of different smaller blocks may be done in the same way (e.g., increase by 1) [0255] 1. Alternatively, the adjustment of corresponding depth of different smaller blocks may be done in the different way (e.g., increase by 1). For example, the adjustment is dependent on block dimension of the smaller block. [0256] 6. QUI-T is not allowed if a split child block cross more than one Virtual pipeline data units (VPDUs). [0257] a. Alternatively, QUI-T is still allowed, however, such child block is forced to be further split until no child block crosses more than one VPDU. [0258] 7. QUI-T is not allowed if the width/height of the current block (or any of the split child block) satisfy some conditions. (Suppose the width and height of the current block are W and H, T1, T2 and T are some integers) [0259] a. QUI-T is not allowed if $W \geq T1$ and $H \geq T2$; [0260] b. QUI-T is not allowed if $W \geq T1$ or $H \geq T2$; [0261] c. QUI-T is not allowed if $W \leq T1$ and $H \leq T2$; [0262] d. QUI-T is not allowed if $W \leq T1$ or $H \leq T2$; [0263] e. QUI-T is not allowed if $W \times H \leq T$; [0264] f. QUI-T is not allowed if $W \times H \geq T$; [0265] g. Horizontal QUI-T is not allowed if $H \leq T$; For example, $T=16$. [0266] h. Horizontal QUI-T is not allowed if $H \geq T$; For example, $T=128$. [0267] i. Vertical QUI-T is not allowed if $W \leq T$; For example, $T=16$. [0268] j. Vertical QUI-T is not allowed if $W \geq T$; For example, $T=128$. [0269] k. T1, T2 and T may be signaled from the encoder to the decoder in VPS/SPS/PPS/picture header/slice header/tile group header/tile header. [0270] l. T1, T2 and T may depend on color components. For example, T1, T2 and T may be different for luma and chroma components. [0271] i. In one example, the signaled thresholds such as T1, T2 and/or T may be shared by QUI-T and TT. [0272] ii. In one example, the signaled thresholds such as T1, T2 and/or T may be shared by QUI-T and BT. [0273] m. T1, T2 and T may depend on whether luma coding tree and chroma coding tree are separated. For example, T1, T2 and T may be different for luma and chroma components if luma coding tree and chroma coding tree are separated. [0274] n. Alternatively, when the transform is not supported for at least one child block due to QUI-T, QUI-T split is invalid. [0275] o. Alternatively, when the depth of one block exceeding the allowed depth for QUI-T splitting, QUI-T split is invalid. [0276] p. Alternatively, when any of a child block size is smaller than the allowed

block size due to QUI-T splitting, QUI-T split is invalid. [0277] 8. QUI-T is allowed if the width/height of the current block (or any of the split child block) satisfy some conditions. (Suppose the width and height of the current block are W and H, T1, T2 and T are some integers) [0278] a. QUI-T is allowed if $W \geq T1$ and $H \geq T2$; [0279] b. QUI-T is allowed if $W \geq T1$ or $H \geq T2$; [0280] c. QUI-T is allowed if $W \leq T1$ and $H \leq T2$; [0281] d. QUI-T is allowed if $W \leq T1$ or $H \leq T2$; [0282] e. QUI-T is allowed if $W \times H \leq T$; [0283] f. QUI-T is allowed if $W \times H \geq T$; [0284] g. Horizontal QUI-T is allowed if $H \leq T$; For example, $T=64$. [0285] h. Horizontal QUI-T is allowed if $H \geq T$; For example, $T=32$. [0286] i. Vertical QUI-T is allowed if $W \leq T$; For example, $T=64$. [0287] j. Vertical QUI-T is allowed if $W \geq T$; For example, $T=32$. [0288] k. T1, T2 and T may be signaled from the encoder to the decoder in VPS/SPS/PPS/picture header/slice header/tile group header/tile header. [0289] l. In one example, the signaled thresholds such as T1, T2 and/or T may be shared by QUI-T and TT. [0290] m. In one example, the signaled thresholds such as T1, T2 and/or T may be shared by QUI-T and BT. [0291] n. T1, T2 and T may depend on color components. For example, T1, T2 and T may be different for luma and chroma components. [0292] o. T1, T2 and T may depend on whether luma coding tree and chroma coding tree are separated. For example, T1, T2 and T may be different for luma and chroma components if luma coding tree and chroma coding tree are separated. [0293] 9. QUI-T is not allowed if the depth of the current block satisfy some conditions. The depth of the current block may refer to QT depth, BT depth, TT depth, QUI-T depth or MTT depth. [0294] a. QUI-T is not allowed if the split depth $\leq T$; [0295] b. QUI-T is not allowed if the split depth $\geq T$; [0296] c. QUI-T is not allowed if the QT split depth $\leq T$; [0297] d. QUI-T is not allowed if the QT split depth $\geq T$; [0298] e. QUI-T is not allowed if the BT split depth $\geq T$; [0299] f. QUI-T is not allowed if the BT split depth $\leq T$; [0300] g. QUI-T is not allowed if the TT split depth $\geq T$; [0301] h. QUI-T is not allowed if the TT split depth $\leq T$; [0302] i. QUI-T is not allowed if the QUI-T split depth $\leq T$; [0303] j. QUI-T is not allowed if the QUI-T split depth $\geq T$; [0304] k. QUI-T is not allowed if the MTT split depth $\leq T$; [0305] l. QUI-T is not allowed if the MTT split depth $\geq T$; [0306] m. T may be signaled from the encoder to the decoder in VPS/SPS/PPS/picture header/slice header/tile group header/tile header. [0307] n. T may depend on color components. For example, T1, T2 and T may be different for luma and chroma components. [0308] o. T may depend on whether luma coding tree and chroma coding tree are separated. For example, T1, T2 and T may be different for luma and chroma components if luma coding tree and chroma coding tree are separated. [0309] 10. QUI-T is allowed if the depth of the current block satisfy some conditions. The depth of the current block may refer to QT depth, BT depth, TT depth, QUI-T depth or MTT depth. [0310] a. QUI-T is allowed if the split depth $\leq T$; [0311] b. QUI-T is allowed if the split depth $\geq T$; [0312] c. QUI-T is allowed if the QT split depth $\leq T$; [0313] d. QUI-T is allowed if the QT split depth $\geq T$; [0314] e. QUI-T is allowed if the BT split depth $\geq T$; [0315] f. QUI-T is allowed if the BT split depth $\leq T$; [0316] g. QUI-T is allowed if the TT split depth $\geq T$; [0317] h. QUI-T is allowed if the TT split depth $\leq T$; [0318] i. QUI-T is allowed if the QUI-T split depth $\leq T$; [0319] j. QUI-T is allowed if the QUI-T split depth $\geq T$; [0320] k. QUI-T is allowed if the MTT split depth $\leq T$; [0321] l. QUI-T is allowed if the MTT split depth $\geq T$; [0322] m. T may be signaled from the encoder to the decoder in VPS/SPS/PPS/picture header/slice header/tile group header/tile header. [0323] n. T may depend on color components. For example, T1, T2 and T may be different for luma and chroma components. [0324] o. T may depend on whether luma coding tree and chroma coding tree are separated. For example, T1, T2 and T may be different for luma and chroma components if luma coding tree and chroma coding tree are separated. [0325] 11. Whether and how to use QUI-T may depend on the position of the current block. For example, whether and how to use QUI-T may depend on the whether the current block crosses the picture/tile/tile group border or not. [0326] a. In one example, vertical QUI-T is not allowed if the current block crosses the picture/tile/tile group bottom border. [0327] b. In one example, horizontal QUI-T is not allowed if the current block crosses the picture/tile/tile group bottom border. [0328] c. In one example, vertical QUI-T is not allowed if the

current block crosses the picture/tile/tile group right border. [0329] d. In one example, horizontal QUI-T is not allowed if the current block crosses the picture/tile/tile group right border. [0330] e. In one example, mixed QUI-T may be not allowed if the current block crosses the picture/tile/tile group right border. [0331] f. In one example, mixed QUI-T may be not allowed if the current block crosses the picture/tile/tile group bottom border. [0332] g. In one example, if a child block split by QUI-T is totally out of the picture/tile/tile group, the child block may be omitted in the encoding/decoding process. [0333] h. In one example, if a child block split by QUI-T is partially out of the picture/tile/tile group, the following may apply [0334] i. The part out of the picture may be omitted in the encoding/decoding process. [0335] ii. The part inside the picture may be further split. [0336] iii. The part inside the picture may be coded as a CU. [0337] 1. Whether the part inside the picture is coded as a CU may depend on the width (w) and height (h) of the part. a. In one example, the part inside the picture may be coded as a CU if $w=2n_{\text{sub}}.w$, $h=2n_{\text{sub}}.h$, where n_w and n_h are integers. [0338] i. In one example, if any child block split by QUI-T is partially/fully out of the picture/tile/tile group, QUI-T is disallowed. [0339] 12. When QUI-T or certain QUI-T pattern is disallowed, the signaling of indication of the usage of the pattern may be also skipped. [0340] a. Alternatively, it may be still signaled but is constrained to be false in a conformance bitstream. [0341] 13. When a child block is split from QUI-T, the child block may not be allowed to be further split with one or more splitting methods as: [0342] a. QT [0343] b. horizontal BT [0344] c. vertical BT [0345] d. horizontal TT [0346] e. vertical TT [0347] f. horizontal UQT [0348] g. vertical UQT [0349] h. QUI-T [0350] It is proposed to QUI-T may be only applied to the leaf nodes, e.g., when one block is not further split according to other partitions. [0351] i. In one example, a flag may be signaled for the leaf node whether to use QUI-T or not. [0352] i. Alternatively, furthermore, indications of which kind QUI-T may be further signaled. [0353] j. Alternatively, indications of disabling QUI-T or which kind QUI-T may be signaled for the leaf node.

Indications of Usage of QUI-T

[0354] 14. Whether to apply QUI-T and/or which kind QUI-T is applied may be signaled from encoder to decoder. [0355] a. In one example, it may be signaled in VPS/SPS/PPS/sequence header/picture header/slice header/tile group header/tile header to indicate whether QUI-T can be applied. [0356] b. In one example, it may be signaled in VPS/SPS/PPS/sequence header/picture header/slice header/tile group header/tile header to indicate which kinds of QUI-T can be applied. [0357] c. In one example, it may be signaled in a block to indicate whether QUI-T is used to split that block. [0358] d. In one example, it may be signaled in a block to indicate which kind of QUI-T is used to split that block. [0359] e. In one example, different QUI-T sets may be designed for different block shapes/sizes. [0360] f. In one example, different QUI-T sets may be designed for pictures/tiles/slices with different temporal layers. [0361] g. In one example, whether or how to apply QUI-T may depend on the video resolution/picture resolution/coded modes/video characteristics (screen content or camera captured sequence or mixed content)/slice type/picture type/tile group type/low delay check flag. [0362] 15. One syntax element may be signaled to indicate no split or partition (including partition tree type and split directions). [0363] a. Alternatively, one syntax element may be firstly signaled to indicate whether to split or not; and another syntax element may be signaled to indicate the partition. [0364] 16. Indication of partition may be represented by two syntax element: selected partition tree type may be firstly signaled, followed by splitting direction if needed. [0365] a. In one example, an index of partition tree type may be signaled in a block to indicate whether a block is split by QT, or QUI-T or non-split. [0366] i. Alternatively, furthermore, the splitting direction (horizontal/vertical/mixed direction) and/or splitting patterns may be further signaled. [0367] b. In one example, an index of partition tree type may be signaled in a block to indicate whether a block is split by BT, or TT, or QUI-T. [0368] i. For example, this index may be conditionally signaled, such as only when at least one of BT, TT and QUI-T is valid for this block. [0369] ii. Alternatively, furthermore, the splitting direction

(horizontal/vertical) and/or splitting patterns may be further signaled. [0370] c. Alternatively, indication of splitting direction may be firstly signaled, followed by partition tree type (such as QT, TT, QUI-T). [0371] i. In one example, a flag is signaled in a block to indicate whether a block is vertical split or horizontal split. The vertical split may be BT vertical split, TT vertical split or QUI-T vertical split. The horizontal split may be BT horizontal split, TT horizontal split or QUI-T horizontal split. [0372] ii. For example, this flag is signaled only when the block is split by BT, or TT, or QUI-T. [0373] iii. For example, this flag is signaled only when both vertical split and horizontal split are valid for this block. [0374] 1. If only vertical split is valid, the flag is not signaled, and horizontal split is inferred to be used. [0375] 2. If only horizontal split is valid, the flag is not signaled, and vertical split is inferred to be used. [0376] d. In one example, a binarized code is signaled in a block to indicate which kind of split (BT, TT, or a kind of QUI-T) is used. In following examples, X represents 0 or 1 and $Y=\sim X$ ($Y=1$ if $X=0$ and $Y=0$ if $X=1$). [0377] i. In one example, the candidate BT, TT or QUI-Ts to be signaled are all vertical splits or horizontal splits depending on previously signaled or derived information. [0378] ii. In one example, a first flag is signaled to indicate whether QUI-T is used. For example, the binarized codewords orderly to represent BT, TT, QUI-T1, QUI-T2, QUI-T3 and QUI-T4 are XX, XY, YXX, YXY, YYX, YYY. [0379] 1. In an alternative example, the binarized codewords orderly to represent BT, TT, QUI-T1 are XX, XY, Y. [0380] iii. In one example, truncated unary code is applied. For example, the binarized codewords orderly to represent BT, TT, QUI-T1, QUI-T2, QUI-T3 and QUI-T4 are X, YX, YYX, YYYYX, YYYYYX, YYYYYY. [0381] iv. In one example, a first flag is signaled to indicate whether BT is used. If BT is not used, then a second flag is signaled to indicate whether QUI-T is used. If QUI-T is used, which kind of QUI-T is used is further signaled. For example, the binarized codewords orderly to represent BT, TT, QUI-T1, QUI-T2, QUI-T3 and QUI-T4 are X, YX, YYXX, YYXY, YYYX, YYYY. [0382] 17. In one example, how to signal which kind of partitions is used in a block may depend on which kinds of partitions (including partition tree type and/or partition directions) are valid for the block. In following examples, X represents 0 or 1 and $Y=\sim X$ ($Y=1$ if $X=0$ and $Y=0$ if $X=1$). [0383] a. In one example, the candidate BT, TT or QUI-Ts to be signaled are all vertical splits or horizontal splits depending on previously signaled or derived information. [0384] b. For example, the non-allowed or invalid split cannot be signaled from the encoder to the decoder, i.e. there is no codeword to represent the non-allowed or invalid split. [0385] c. In one example, if there is only one kind of split from BT, TT and QUI-Ts is valid, then the binarized code to indicate which kind of split (BT, TT, or a kind of QUI-T) is used is not signaled. [0386] d. In one example, if there are only two kinds of split from BT, TT and QUI-Ts are valid, then a flag is signaled to indicate which one of the two valid splits is used. [0387] e. In one example, the code to indicate which kind of split (BT, TT, or a kind of QUI-T) is binarized as a truncated unary code. [0388] i. For example, the maximum value of the truncated unary code is $N-1$, where N is the number of valid splits (BT, TT and QUI-Ts). [0389] ii. For example, no codeword represents an invalid split. In other words, the invalid split is skipped when building the codeword table. [0390] f. In one example, if no QUI-T is valid, the flag indicating whether QUI-T is used is not signaled and inferred to be false. For example, the binarized codewords orderly to represent BT and TT are X and Y. [0391] g. In one example, if only one kind of QUI-T is valid and QUI-T is signaled to be used, then no further information is signaled to indicate which QUI-T is used. The valid QUI-T is used implicitly. [0392] h. In one example, if only two kinds of QUI-T are valid and QUI-T is signaled to be used, then a flag is signaled to indicate which QUI-T is used. [0393] i. In one example, if only three kinds of QUI-T are valid and QUI-T is signaled to be used, then a message is signaled to indicate which QUI-T is used. For example, the binarized codewords orderly to represent the three QUI-Ts are X, YX, YY. [0394] j. In one example, the binarization and/or signaling method is not changed according to which kinds of split is valid in the block. An invalid split cannot be chosen in a conformance bit-stream. [0395] 18. Indications of partition may be coded by arithmetic coding with one or multiple contexts. [0396] a. In one example, only partial

bins of a bin string may be coded with contexts and remaining bins may be coded with bypass mode (i.e., no context is utilized). [0397] b. Alternatively, all bins of a bin string may be coded with contexts. [0398] c. Alternatively, all bins of a bin string may be coded with bypass mode. [0399] d. For a bin coded with context, one or multiple contexts may be used. [0400] e. The context may depend on: [0401] i. The position or index of the bin. [0402] ii. The partitioning of spatial/temporal neighboring blocks. [0403] iii. The current partition depth (e.g., QT depth/BT depth/TT depth/QUI-T depth/MTT depth) of current block. [0404] iv. The partition depth (e.g., QT depth/BT depth/TT depth/QUI-T depth/MTT depth) of spatial/temporal neighboring blocks and/or spatial/temporal non-adjacent blocks. [0405] v. The coding modes of spatial/temporal neighboring blocks. [0406] vi. The width/height of spatial/temporal neighboring blocks. [0407] vii. The width/height of the current block [0408] viii. Slice types/picture types/tile group type [0409] ix. Color component [0410] x. Statistical results of partition types from previously coded blocks [0411] 19. Whether and/or how to use QUI-T may depend on color format (such as 4:4:4 or 4:2:0) and/or color components. [0412] a. Whether and how to use QUI-T may depend on whether luma and chroma coding trees are separated. [0413] b. In one example, QUI-T can only be applied on luma component when luma and chroma coding trees are separated. [0414] 20. The above methods may be also applicable to SnT, StT, OctT, UQT.

[0415] The examples described above may be incorporated in the context of the method described below, e.g., method **1300**, which may be implemented at a video decoder/encoder.

[0416] FIG. **13** illustrates a flowchart of an exemplary method for video processing. The method **1300** comprises, at step **1310**, determining, for a current video block, whether a first partition mode is applicable to the current video block in responsive to at least one condition, wherein the current video block is split into M sub-blocks in the first partition mode, and $M > 4$; and at step **1320**, performing a conversion for the current video block based on the determination.

[0417] FIG. **14** illustrates a flowchart of an exemplary method for video processing. The method **1400** comprises: determining, at step **1410**, for a video block, whether and/or how to apply a first partition mode to the video block based on an indication, wherein the video block is split into M portions in the first partition mode, $M > 4$; and performing a conversion of the video block based on the determination.

[0418] Some embodiments and techniques related to methods **1300** and **1400** may be described using the following examples.

[0419] In an example, there is disclosed a method for video processing, comprising: determining, for a current video block, whether a first partition mode is applicable to the current video block in responsive to at least one condition, wherein the current video block is split into M sub-blocks in the first partition mode, and $M > 4$; and performing a conversion for the current video block based on the determination.

[0420] In an example, $M = 5$, and the current video block is split into five sub-blocks using Quinary-Tree (QUI-T) in the first partition mode.

[0421] In an example, the at least one condition depends on at least one of a maximum block size, a minimum block size, and a maximum depth allowed for the first partition mode.

[0422] In an example, the maximum depth comprises at least one of a maximum bit depth and a maximum split depth.

[0423] In an example, the at least one condition is determined from an indication signaled in a video unit level wherein the video unit comprises at least one of sequence, video, picture, slice, tile group, a coding tree unit (CTU) row or a CTU region.

[0424] In an example, the at least one condition is determined from an indication signaled in at least one of a sequence parameter set (SPS), a video parameter set (VPS), a picture parameter set (PPS), an adaptation parameter set (APS), a sequence header, a picture header, a slice header, a tile group header.

[0425] In an example, the at least one condition is derived from at least one of a depth of a

multiple-type tree (MTT) or a depth of quadtree (QT).

[0426] In an example, the maximum block size is a size of a largest coding block/unit or a size of a virtual pipeline data unit (VPDU).

[0427] In an example, the largest coding block/unit is a coding tree block/unit (CTB/CTU).

[0428] In an example, the at least one condition depends on at least one of a profile, level and tier of a coding standard.

[0429] In an example, at least one of the maximum block size, minimum block size, and maximum depth is derived in a same way as that of QT partition mode.

[0430] In an example, the at least one condition depends on at least one of a tile group, a tile, a slice type, a color component and an activation of a dual tree

[0431] In an example, the first partition mode has a plurality of partition patterns by which one block can be split into M sub-blocks in different ways, and the at least one condition differs between different partition patterns.

[0432] In an example, all sub-blocks are leaf nodes, and the method further comprises: splitting at least one sub-block into a plurality of portions singly or recursively.

[0433] In an example, the at least one sub-block is split in at least one of the first partition mode, a QT partition mode and an MTT partition mode.

[0434] In an example, a depth of the at least one sub-block is adjusted based on a depth of the current video block.

[0435] In an example, the depth of the at least one sub-block is equal to the depth of the current video block plus 1.

[0436] In an example, depths of all sub-blocks are adjusted in a same way or depths of different sub-blocks are adjusted in different ways.

[0437] In an example, a depth of the at least one sub-block is adjusted based on a size of the at least one sub-block.

[0438] In an example, the at least one condition depends on a position of each sub-block.

[0439] In an example, the first partition mode is not applicable to the current video block if at least one sub-block crosses a border of a VPDU.

[0440] In an example, the method further comprises: splitting any sub-block which crosses a border of a VPDU into a plurality of portions singly or recursively until no portion crosses the border of the VPDU.

[0441] In an example, the at least one condition depends on a dimension of the current video block.

[0442] In an example, the first partition mode is applicable to the current video block if the dimension of the current video block satisfies at least one of:

$W \geq T1$;

$H \geq T2$;

$W \times H \geq T3$;

[0443] wherein W, H represent a width and height of the current video block respectively, and T1, T2 and T3 represents first to third thresholds respectively.

[0444] In an example, the first partition mode is applicable to the current video block if the dimension of the current video block satisfies at least one of:

$W \leq T1'$;

$H \leq T2'$;

$W \times H \leq T3'$;

[0445] wherein W, H represent a width and height of the current video block respectively, and T1', T2' and T3' represents first to third thresholds respectively.

[0446] In an example, the first partition mode is applicable to the current video block in a horizontal direction if the dimension of the current video block satisfies at least one of:

$H \leq T4$;

$H \geq T5$;

[0447] wherein T4 and T5 represents fourth and fifth thresholds respectively.

[0448] In an example, T4=64, and T5=32.

[0449] In an example, the first partition mode is applicable to the current video block in a vertical direction if the dimension of the current video block satisfies at least one of:

$W \leq T6$;

$W \geq T7$;

[0450] wherein T6 and T7 represents sixth and seventh thresholds respectively.

[0451] In an example, T6=64, and T7=32.

[0452] In an example, the first partition mode is not applicable to the current video block in a horizontal direction if the dimension of the current video block satisfies at least one of:

$H \leq 16$;

$H \geq 128$.

[0453] In an example, the first partition mode is not applicable to the current video block in a vertical direction if the dimension of the current video block satisfies at least one of:

$W \leq 16$;

$W \geq 128$.

[0454] In an example, the at least one condition depends on a split depth of the current video block.

[0455] In an example, the first partition mode is applicable to the current video block if the split depth of the current video block satisfies at least one of:

$D \leq D1$;

$D \geq D2$;

[0456] wherein D, D1 and D2 represent the split depth of the current video block, a first depth threshold, and a second depth threshold respectively.

[0457] In an example, the split depth of the current video block comprises at least one of QT split depth, binary tree (BT) split depth, ternary tree (TT) split depth, MTT split depth and split depth in the first partition mode.

[0458] In an example, the at least one condition depends on a position of the current video block.

[0459] In an example, the first partition mode is not applicable to the current video block in at least one direction if the current video block crosses a border of at least one of a picture, tile, and tile group comprising the current video block.

[0460] In an example, the border comprises at least one of a bottom border and a right border.

[0461] In an example, the at least one direction comprises a vertical direction, a horizontal direction, and a mixed direction including both vertical and horizontal directions.

[0462] In an example, the method further comprises: skipping any sub-block, which is located

outside one of a picture, tile, and tile group comprising the current video block, in a subsequent conversion.

[0463] In an example, if any sub-block has first and second portions which are located outside and inside one of a picture, tile and tile group respectively, the method comprises skipping the first portion in a subsequent conversion.

[0464] In an example, the second portion is split into a plurality of sub-portions.

[0465] In an example, the second portion is converted as a coding unit.

[0466] In an example, at least one of width and height of the second portion is equal to a power of 2.

[0467] In an example, the first partition mode is not applicable to the current video blocks if any sub-block is partial or fully outside at least one of a picture, tile, and tile group.

[0468] In an example, the first partition mode is not applicable to the current video block if one of the following is satisfied: [0469] the maximum depth allowed for the first partition mode is reached for at least one sub-block; [0470] the minimum block size allowed for the first partition mode is reached for at least one sub-block; [0471] a block size for which a transform can be supported is reached for at least one sub-block;

[0472] In an example, at least one of thresholds is signaled in a sequence parameter set (SPS), a video parameter set (VPS), a picture parameter set (PPS), a picture header, a slice header, a tile group header or a tile header.

[0473] In an example, at least one of thresholds is signaled in a video unit level wherein the video unit comprises at least one of sequence, video, picture, slice, tile group, a coding tree unit (CTU) row or a CTU region.

[0474] In an example, the signaled at least one of thresholds is shared by the ternary tree (TT) partition and the first partition mode or shared by the binary tree (BT) partition and the first partition mode.

[0475] In an example, at least one threshold depends on sample components of the current video block.

[0476] In an example, the sample components comprise at least one of color components, luma component and chroma components.

[0477] In an example, at least one threshold differs between the luma component and the chroma components if luma and chroma coding trees of the current video block are separated from each other.

[0478] In an example, at least one of the following partition modes is not available for at least one of sub-blocks: [0479] QT partition; [0480] BT partition in a horizontal direction; [0481] BT partition in a vertical direction; [0482] TT partition in a horizontal direction; [0483] TT partition in a vertical direction; [0484] unsymmetrical quadtree (UQT) partition in a horizontal direction; [0485] UQT partition in a vertical direction; and [0486] the first partition mode.

[0487] In an example, the at least one condition depends on whether the current video block belongs to a leaf node to which any other partition modes is not applicable.

[0488] In an example, the method further comprises: determining a first indication indicating whether the first partition mode is applicable to the leaf node.

[0489] In an example, the first partition mode has one or more partition patterns, and the method comprises: determining a second indication indicating which partition pattern is used.

[0490] In an example, the at least one condition depends on at least one of color format, luma component and chroma components.

[0491] In an example, the first partition mode is only applicable to luma components of the current video block if luma and chroma coding trees of the current video block are separated from each other.

[0492] In another aspect, there is disclosed a method for video processing, comprising: determining, for a video block, whether and/or how to apply a first partition mode to the video

block based on an indication, wherein the video block is split into M portions in the first partition mode, $M > 4$; and performing a conversion of the video block based on the determination.

[0493] In an example, $M=5$, and the video block is split into five portions using Quinary-Tree (QUI-T) in the first partition mode.

[0494] In an example, the first partition mode has one or more partition patterns.

[0495] In an example, the indication is signaled in one of a sequence parameter set (SPS), a video parameter set (VPS), a picture parameter set (PPS), a sequence header, a picture header, a slice header, a tile group header or a tile header.

[0496] In an example, the indication is signaled in a video unit level wherein the video unit comprises at least one of sequence, video, picture, slice, tile group, a coding tree unit (CTU) row or a CTU region

[0497] In an example, the indication is signaled in a bitstream representation of the video block.

[0498] In an example, multiple partition patterns are designed based on a shape or size of the video block.

[0499] In an example, multiple partition patterns are designed based on one of pictures, tiles and slices with different temporal layers.

[0500] In an example, the indication depends on at least one of a video resolution, picture resolution, coding mode, video content, slice type, picture type, tile group type, low delay check flag.

[0501] In an example, the video content comprises at least one of a screen content, a camera captured sequence or mixed content.

[0502] In an example, the indication is represented by one or more syntax elements, wherein the one or more syntax elements comprise a first syntax element which indicates whether a split is performed on the video block.

[0503] In an example, the one or more syntax elements further comprises a second syntax element which indicates a partition tree and a partition direction to be used in the split.

[0504] In an example, the indication is represented by one or more syntax elements, wherein the one or more syntax elements comprises a first syntax element which indicates an index of a type of a partition tree to be applied to the video block.

[0505] In an example, the type of the partition tree belongs to at least one of: BT, TT, QT, the first partition and non-split.

[0506] In an example, the one or more syntax elements further comprises a second syntax element which indicates at least one of a partition direction and a partition pattern.

[0507] In an example, the first syntax element is signaled only if a corresponding partition tree is valid for the video block.

[0508] In an example, the partition direction comprises at least one of a vertical direction, a horizontal direction, and a mixed direction including both vertical and horizontal directions.

[0509] In an example, the second syntax element is signaled prior to or subsequent to the first syntax element.

[0510] In an example, the second syntax element is signaled only if a corresponding partition direction is valid for the video block.

[0511] In an example, the indication is represented as a binarized codeword which comprises a first bin to indicate whether the first partition mode is applied to the video block.

[0512] In an example, if the first bin indicates that the first partition mode is not applied, the binarized codeword further comprises a second bin to indicate whether BT or TT partition is applied to the video block.

[0513] In an example, if the first bin indicates that first partition mode is applied, the binarized code further comprises one or more bins to indicate which partition pattern is applied to the video block.

[0514] In an example, the indication is represented as a binarized codeword, wherein [0515] the

binarized codeword uses a first bin to indicate one of the BT and TT partitions; [0516] the binarized codeword uses first and second bins to indicate the other one of the BT and TT partitions; and [0517] the binarized codeword uses first, second, and one or more subsequent bins to indicate which partition pattern of the first partition mode.

[0518] In an example, the indication is represented as a binarized codeword, wherein [0519] the binarized codeword uses a first bin to indicate the first partition mode; [0520] the binarized codeword uses first and second bins to indicate the BT and TT partitions.

[0521] In an example, the binarized codeword is a truncated unary code.

[0522] In an example, the indication is signaled for one or more partition modes which are valid for the video block, and the one or more partition modes comprise at least one of the BT, TT and first partition modes.

[0523] In an example, the one or more partition modes use only one partition direction which is previously signaled or derived.

[0524] In an example, no indication is signaled for any partition mode which is invalid for the video block or for only one valid partition mode of the video block.

[0525] In an example, if there are only two partition modes valid for the video block, the indication comprises a flag to indicate which partition mode is used for the video block.

[0526] In an example, if there are more than two partition modes valid for the video block, the indication comprises a binarized codeword to indicate which partition mode is used for the video block.

[0527] In an example, the binarized codeword is a truncated unary code, and the truncated unary code has a maximum value equal to $N-1$, wherein N represents an amount of the partition modes valid for the video block.

[0528] In an example, if the first partition mode or at least one partition pattern of the first partition mode is invalid for the video block, a signaling of the at least one partition pattern is skipped or a flag for the signaling is constrained to be false.

[0529] In an example, if the first partition mode is applied to the video block, the indication further indicates which partition pattern is applied to the video block.

[0530] In an example, if the first partition mode has only one partition pattern valid for the video block, the partition pattern is implicitly used without being signaled.

[0531] In an example, if the first partition mode has two partition patterns valid for the video block, the indication comprises a flag to indicate which partition pattern is used for the video block.

[0532] In an example, if the first partition mode has more than two partition patterns valid for the video block, the indication comprises a binarized codeword to indicate which partition pattern is used for the video block.

[0533] In an example, the binarized codeword is coded by an arithmetic coding with at least one context or with a bypass mode.

[0534] In an example, partial bins of the binarized codeword are coded with the at least one context, and remaining bins are coded with the bypass mode.

[0535] In an example, [0536] all bins of the binarized codeword are coded with the at least one context; or [0537] all bins of the binarized codeword are coded with the bypass mode.

[0538] In an example, the at least one context depends on at least one of: [0539] a position or index of a bin in the binarized codeword; [0540] characteristic of at least one of spatial neighbouring block, temporal neighbouring block, and the video block; [0541] statistical results of partition types from previously coded blocks; [0542] a type of at least one of slice, picture and title group covering the video block; and color component.

[0543] In an example, the characteristic of at least one of spatial block, temporal neighbouring block, and the video block comprises at least one of: [0544] a partition mode; [0545] a partition depth; [0546] a coding mode; [0547] a width; and [0548] a height.

[0549] In an example, $M=5, 6, 7$, or 8 .

[0550] In an example, the first partition mode comprises a unsymmetrical quad-tree (UQT) partition.

[0551] In an example, the conversion includes encoding the current video block into the bitstream representation of a video and decoding the current video block from the bitstream representation of the video.

[0552] In an example, there is disclosed an apparatus in a video system comprising a processor and a non-transitory memory with instructions thereon, wherein the instructions upon execution by the processor, cause the processor to implement the method in any one of examples described above.

[0553] In an example, there is disclosed a computer program product stored on a non-transitory computer readable media, the computer program product including program code for carrying out the method in any one of examples described above.

5. Example Implementations of the Disclosed Technology

[0554] FIG. 12 is a block diagram of a video processing apparatus 1200. The apparatus 1200 may be used to implement one or more of the methods described herein. The apparatus 1200 may be embodied in a smartphone, tablet, computer, Internet of Things (IoT) receiver, and so on. The apparatus 1200 may include one or more processors 1202, one or more memories 1204 and video processing hardware 1206. The processor(s) 1202 may be configured to implement one or more methods (including, but not limited to, method 1200) described in the present disclosure. The memory (memories) 1204 may be used for storing data and code used for implementing the methods and techniques described herein. The video processing hardware 1206 may be used to implement, in hardware circuitry, some techniques described in the present disclosure.

[0555] In some embodiments, the video coding methods may be implemented using an apparatus that is implemented on a hardware platform as described with respect to FIG. 12.

[0556] From the foregoing, it will be appreciated that specific embodiments of the presently disclosed technology have been described herein for purposes of illustration, but that various modifications may be made without deviating from the scope of the disclosure. Accordingly, the presently disclosed technology is not limited except as by the appended claims.

[0557] Implementations of the subject matter and the functional operations described in the present disclosure can be implemented in various systems, digital electronic circuitry, or in computer software, firmware, or hardware, including the structures disclosed in this specification and their structural equivalents, or in combinations of one or more of them. Implementations of the subject matter described in this specification can be implemented as one or more computer program products, i.e., one or more modules of computer program instructions encoded on a tangible and non-transitory computer readable medium for execution by, or to control the operation of, data processing apparatus. The computer readable medium can be a machine-readable storage device, a machine-readable storage substrate, a memory device, a composition of matter effecting a machine-readable propagated signal, or a combination of one or more of them. The term “data processing unit” or “data processing apparatus” encompasses all apparatus, devices, and machines for processing data, including by way of example a programmable processor, a computer, or multiple processors or computers. The apparatus can include, in addition to hardware, code that creates an execution environment for the computer program in question, e.g., code that constitutes processor firmware, a protocol stack, a database management system, an operating system, or a combination of one or more of them.

[0558] A computer program (also known as a program, software, software application, script, or code) can be written in any form of programming language, including compiled or interpreted languages, and it can be deployed in any form, including as a stand-alone program or as a module, component, subroutine, or other unit suitable for use in a computing environment. A computer program does not necessarily correspond to a file in a file system. A program can be stored in a portion of a file that holds other programs or data (e.g., one or more scripts stored in a markup language document), in a single file dedicated to the program in question, or in multiple

coordinated files (e.g., files that store one or more modules, sub programs, or portions of code). A computer program can be deployed to be executed on one computer or on multiple computers that are located at one site or distributed across multiple sites and interconnected by a communication network.

[0559] The processes and logic flows described in this specification can be performed by one or more programmable processors executing one or more computer programs to perform functions by operating on input data and generating output. The processes and logic flows can also be performed by, and apparatus can also be implemented as, special purpose logic circuitry, e.g., a field programmable gate array (FPGA) or an application specific integrated circuit (ASIC).

[0560] Processors suitable for the execution of a computer program include, by way of example, both general and special purpose microprocessors, and any one or more processors of any kind of digital computer. Generally, a processor will receive instructions and data from a read only memory or a random access memory or both. The essential elements of a computer are a processor for performing instructions and one or more memory devices for storing instructions and data.

Generally, a computer will also include, or be operatively coupled to receive data from or transfer data to, or both, one or more mass storage devices for storing data, e.g., magnetic, magneto optical disks, or optical disks. However, a computer need not have such devices. Computer readable media suitable for storing computer program instructions and data include all forms of nonvolatile memory, media and memory devices, including by way of example semiconductor memory devices, e.g., erasable programmable read-only memory (EPROM), electrically erasable programmable read-only memory (EEPROM), and flash memory devices. The processor and the memory can be supplemented by, or incorporated in, special purpose logic circuitry.

[0561] It is intended that the specification, together with the drawings, be considered exemplary only, where exemplary means an example. As used herein, the singular forms “a”, “an” and “the” are intended to include the plural forms as well, unless the context clearly indicates otherwise. Additionally, the use of “or” is intended to include “and/or”, unless the context clearly indicates otherwise.

[0562] While the present disclosure contains many specifics, these should not be construed as limitations on the scope of any embodiment or of what may be claimed, but rather as descriptions of features that may be specific to particular embodiments of particular inventions. Certain features that are described in the present disclosure in the context of separate embodiments can also be implemented in combination in a single embodiment. Conversely, various features that are described in the context of a single embodiment can also be implemented in multiple embodiments separately or in any suitable subcombination. Moreover, although features may be described above as acting in certain combinations and even initially claimed as such, one or more features from a claimed combination can in some cases be excised from the combination, and the claimed combination may be directed to a subcombination or variation of a subcombination.

[0563] Similarly, while operations are depicted in the drawings in a particular order, this should not be understood as requiring that such operations be performed in the particular order shown or in sequential order, or that all illustrated operations be performed, to achieve desirable results. Moreover, the separation of various system components in the embodiments described in the present disclosure should not be understood as requiring such separation in all embodiments.

[0564] Only a few implementations and examples are described and other implementations, enhancements and variations can be made based on what is described and illustrated in the present disclosure.

Claims

1. A method for video processing, comprising: determining, for a conversion between a current video block of a video and a bitstream of the video, whether a first partition mode is applied to the

current video block in responsive to at least one condition, wherein the current video block is split into M sub-blocks in the first partition mode, and $M > 4$; and performing the conversion based on the determining.

2. The method of claim 1, wherein the at least one condition is determined from an indication signaled in a video unit level, and wherein the video unit level comprises at least one of sequence, video, picture, slice, tile group, a coding tree unit (CTU) row or a CTU region, or the at least one condition is determined from an indication signaled in at least one of a sequence parameter set (SPS), a video parameter set (VPS), a picture parameter set (PPS), an adaptation parameter set (APS), a sequence header, a picture header, a slice header, a tile group header.

3. The method of claim 1, wherein the first partition mode has a plurality of partition patterns by which one block can be split into M sub-blocks in different ways, and the at least one condition differs between different partition patterns, wherein the at least one condition depends on at least one of a profile, level, and tier of a coding standard; or the at least one condition depends on at least one of a tile group, a tile, a slice type, a color component and an activation of a dual tree; or the at least one condition depends on a position of each sub-block; or the at least one condition depends on a dimension of the current video block; or the at least one condition depends on at least one of color format, luma component and chroma components, and wherein multiple partition patterns are designed based on at least one of: a shape or size of the current video block, or one of pictures, tiles and slices with different temporal layers.

4. The method of claim 1, wherein all sub-blocks are leaf nodes, and the method further comprises splitting at least one sub-block into a plurality of portions singly or recursively, wherein the at least one sub-block is split in at least one of the first partition mode, a quad tree (QT) partition mode and an multi-type tree (MTT) partition mode, wherein a depth of the at least one sub-block is adjusted based on a depth of the current video block or based on a size of the at least one sub-block, or the depth of the at least one sub-block is equal to the depth of the current video block plus 1, and wherein depths of all sub-blocks are adjusted in a same way or depths of different sub-blocks are adjusted in different ways.

5. The method of claim 1, wherein the first partition mode is not applied to the current video block when at least one sub-block crosses a border of a virtual pipeline data unit (VPDU), and the method further comprises: splitting any sub-block which crosses a border of a VPDU into a plurality of portions singly or recursively until no portion crosses the border of the VPDU.

6. The method of claim 5, wherein whether the first partition mode is applied to the current video block is determined based on one or more of the following conditions: the first partition mode is applicable to the current video block when a dimension of the current video block satisfies at least one of:

$W \geq T1$;

$H \geq T2$;

$W \times H \geq T3$; wherein W, H represent a width and height of the current video block respectively, and T1, T2 and T3 represents first to third thresholds respectively; or the first partition mode is applicable to the current video block when the dimension of the current video block satisfies at least one of:

$W \leq T1'$;

$H \leq T2'$;

$W \times H \leq T3'$; wherein W, H represent a width and height of the current video block respectively, and T1', T2' and T3' represents first to third thresholds, respectively; or the first partition mode is applicable to the current video block in a horizontal direction when the dimension of the current video block satisfies at least one of:

$H \leq T4$;

$H \geq T5$; wherein T4 and T5 represents fourth and fifth thresholds respectively, wherein $T4 = 64$, and $T5 = 32$; or the first partition mode is applicable to the current video block in a vertical direction

when the dimension of the current video block satisfies at least one of:

$W \leq T6$;

$W \geq T7$; wherein $T6$ and $T7$ represents sixth and seventh thresholds respectively, wherein $T6=64$, and $T7=32$; or the first partition mode is not applicable to the current video block in the horizontal direction when the dimension of the current video block satisfies at least one of:

$H \leq 16$;

$H \geq 128$; or the first partition mode is not applicable to the current video block in the vertical direction when the dimension of the current video block satisfies at least one of:

$W \leq 16$;

$W \geq 128$; or the first partition mode is not applicable to the current video block when any sub-block is partial or fully outside at least one of a picture, tile, and tile group.

7. The method of claim 3, wherein in a case where the at least one condition depends on a position of each sub-block, the method further comprises: skipping any sub-block, which is located outside one of a picture, tile, and tile group comprising the current video block, in a subsequent conversion, wherein when any sub-block has first and second portions which are located outside and inside one of a picture, tile, and tile group respectively, the method comprises skipping the first portion in the subsequent conversion, wherein the second portion is split into a plurality of sub-portion or is converted as a coding unit, and wherein at least one of width and height of the second portion is equal to a power of 2.

8. The method of claim 1, wherein in a case where the at least one condition depends on at least one of color format, luma component and chroma components, the first partition mode is only applicable to luma components of the current video block when luma and chroma coding trees of the current video block are separated from each other.

9. The method of claim 1, wherein whether the first partition mode is applied to the current video block is determined based on an indication, wherein the indication depends on at least one of a video resolution, picture resolution, coding mode, video content, slice type, picture type, tile group type, low delay check flag, wherein the video content comprises at least one of a screen content, a camera captured sequence or mixed content.

10. The method of claim 9, wherein the indication is represented by one or more syntax elements, wherein the one or more syntax elements comprise a first syntax element which indicates whether a split is performed on the current video block, and wherein the one or more syntax elements further comprise a second syntax element which indicates a partition tree and a partition direction to be used in the split; or wherein the indication is represented by one or more syntax elements, wherein the one or more syntax elements comprise a first syntax element which indicates an index of a type of a partition tree to be applied to the current video block, and the type of the partition tree belongs to at least one of: binary tree (BT), ternary tree (TT), quad tree (QT), a first partition and non-split, wherein the one or more syntax elements further comprise a second syntax element which indicates at least one of a partition direction and a partition pattern, and wherein the partition direction comprises at least one of a vertical direction, a horizontal direction, and a mixed direction including both vertical and horizontal directions.

11. The method of claim 10, wherein the first syntax element is signaled only when a corresponding partition tree is valid for the current video block, and the second syntax element is signaled only when a corresponding partition direction is valid for the current video block, and wherein the second syntax element is signaled prior to or subsequent to the first syntax element.

12. The method of claim 9, wherein the indication is represented as a binarized codeword which comprises a first bin to indicate whether the first partition mode is applied to the current video block, in which when the first bin indicates that the first partition mode is not applied, the binarized codeword further comprises a second bin to indicate whether binary tree (BT) or ternary tree (TT) partition is applied to the current video block, and when the first bin indicates that first partition mode is applied, wherein the binarized code further comprises one or more bins to indicate which

partition pattern is applied to the current video block; or wherein the indication is represented as a binarized codeword, the binarized codeword uses a first bin to indicate one of the BT and TT partitions, and the binarized codeword uses first and second bins to indicate another one of the BT and TT partitions, and wherein the binarized codeword uses first, second, and one or more subsequent bins to indicate which partition pattern of the first partition mode, and wherein the binarized codeword is a truncated unary code.

13. The method of claim 9, wherein the indication is signaled for one or more partition modes which are valid for the current video block, and no indication is signaled for any partition mode which is invalid for the current video block or for only one valid partition mode of the current video block, wherein the one or more partition modes comprise at least one of binary tree (BT), ternary tree (TT), and first partition modes, and wherein the one or more partition modes use only one partition direction which is previously signaled or derived, and wherein when there are only two partition modes valid for the current video block, the indication comprises a flag to indicate which partition mode is used for the current video block, or comprises a binarized codeword to indicate which partition mode is used for the current video block, and the binarized codeword is a truncated unary code, and the truncated unary code has a maximum value equal to $N-1$, and wherein N represents an amount of the partition modes valid for the current video block.

14. The method of claim 13, wherein when a first partition mode is applied to the current video block, the indication further indicates which partition pattern is applied to the current video block, and one of the following conditions is applied: when the first partition mode has only one partition pattern valid for the current video block, the partition pattern is implicitly used without being signaled; or when the first partition mode has two partition patterns valid for the current video block, the indication comprises a flag to indicate which partition pattern is used for the current video block; or when the first partition mode has more than two partition patterns valid for the current video block, the indication comprises a binarized codeword to indicate which partition pattern is used for the current video block.

15. The method of claim 12, wherein the binarized codeword is coded by an arithmetic coding with at least one context or with a bypass mode, and one of the following conditions is applied: partial bins of the binarized codeword are coded with the at least one context, and remaining bins are coded with the bypass mode; or all bins of the binarized codeword are coded with the at least one context; or all bins of the binarized codeword are coded with the bypass mode, and wherein the at least one context depends on at least one of: a position or index of a bin in the binarized codeword; or characteristic of at least one of spatial neighbouring block, temporal neighbouring block, and the current video block, wherein the characteristic of at least one of spatial block, temporal neighbouring block, and the current video block comprises at least one of: a partition mode; a partition depth; a coding mode; a width; and a height; or statistical results of partition types from previously coded blocks; or a type of at least one of slice, picture and title group covering the current video block; or color component.

16. The method of claim 1, wherein at least one of the M sub-blocks has a width less than the width of the current video block or has a height less than the height of the current video block, wherein the current video block is split in only one of a horizontal direction and a vertical direction, and each of M sub-blocks has a same height as the current video block when the current video block is split only in the vertical direction, and each of M sub-blocks has a same width, and wherein none of M sub-blocks can be further split.

17. The method of claim 1, wherein the conversion includes encoding the current video block into the bitstream.

18. The method of claim 1, wherein the conversion includes decoding the current video block from the bitstream.

19. An apparatus for processing video data comprising a processor and a non-transitory memory with instructions thereon, wherein the instructions upon execution by the processor, cause the

processor to: determine, for a conversion between a current video block of a video and a bitstream of the video, whether a first partition mode is applied to the current video block in responsive to at least one condition, wherein the current video block is split into M sub-blocks in the first partition mode, and $M > 4$; and perform the conversion based on the determining.

20. A non-transitory computer-readable recording medium storing a bitstream of a video which is generated by a method performed by a video processing apparatus, wherein the method comprises: determining, for a current video block of the video, whether a first partition mode is applied to the current video block in responsive to at least one condition, wherein the current video block is split into M sub-blocks in the first partition mode, and $M > 4$; and generating the bitstream of the video based on the determining.
